

Modul Kuliah – IF231303

Arsitektur Perangkat Lunak

Alfa Yohannis

**Program Studi Teknologi Informasi
Universitas Pradita**

Daftar Isi

1	Pengenalan Arsitektur Perangkat Lunak dan Client-Server	5
1.1	Pendahuluan	5
1.2	Aspek yang Dinilai pada Sebuah Arsitektur	6
1.3	Studi Kasus Singkat	6
1.4	Arsitektur Client-Server	7
1.5	Dua Biaya yang Melekat	8
1.6	Kelebihan dan Kekurangan	9
1.7	Ringkasan	9
1.8	Latihan	10
2	Layered Architecture	15
2.1	Pendahuluan	15
2.2	Pembagian <i>Layer</i>	16
2.3	Empat <i>Layer</i> sebagai Empat Kelas	18
2.4	Aturan Ketergantungan	19
2.5	<i>Closed Layer</i> dan <i>Open Layer</i>	19
2.6	Kapan Jalan Pintas Dapat Dibenarkan	20
2.7	Kelebihan dan Kekurangan	23
2.8	Ringkasan	23
2.9	Latihan	24
3	Model-View-*	31
3.1	Pendahuluan	31
3.2	Peran Ketiga Komponen	32
3.3	Empat Varian dan Arah Ketergantungannya	32
3.3.1	Model-View-Controller	33
3.3.2	Model-View-Presenter	37
3.3.3	Model-View-ViewModel	38
3.3.4	Model-View-Intent	39
3.4	Produk yang Memakai Setiap Varian	42

3.5	Ketergantungan View sebagai Pembeda	44
3.6	Ketika Satu Komponen Menggemuk	45
3.6.1	Fat Controller	45
3.6.2	Fat Model	46
3.6.3	Fat View	46
3.7	Kelebihan dan Kekurangan	47
3.8	Ringkasan	48
3.9	Latihan	48
4	Ports and Adapters	53
4.1	Pendahuluan	53
4.2	Port dan Adapter	54
4.3	Menerima Adapter sebagai Argumen	56
4.4	Tiga Akibat yang Dapat Diukur	60
4.5	Kelebihan dan Kekurangan	60
4.6	Ringkasan	61
4.7	Latihan	62
	Daftar Pustaka	65

Bab 1

Pengenalan Arsitektur Perangkat Lunak dan Client-Server

Tujuan Pembelajaran

1. Menjelaskan arti arsitektur perangkat lunak beserta aspek yang membedakannya dari rancangan rinci.
2. Membedakan tanggung jawab klien dari tanggung jawab server pada arsitektur client-server.
3. Mengukur waktu bolak-balik satu permintaan dan perubahannya ketika jumlah klien bertambah.

1.1 Pendahuluan

Setiap sistem perangkat lunak punya arsitektur, baik dirancang maupun tidak. Arsitektur adalah susunan bagian-bagian besar sistem beserta aturan hubungan di antaranya. Susunan itu menentukan hal-hal yang mahal diubah di kemudian hari, misalnya batas antar modul dan cara modul berkomunikasi.

Yang membedakan arsitektur dari rancangan rinci adalah biaya perubahannya. Mengganti nama sebuah fungsi murah. Memindahkan aturan bisnis dari satu bagian ke bagian lain juga masih murah. Sebaliknya, memecah satu sistem menjadi beberapa layanan terpisah menyentuh hampir seluruh berkas. Keputusan yang mahal diubah itulah yang disebut keputusan arsitektural.

Bab ini membuka dengan pengertian arsitektur perangkat lunak beserta aspek yang dinilainya, lalu menutup dengan bentuk pertama yang dibahas dalam modul ini, yaitu client-server. Bentuk tersebut dipilih lebih dahulu karena hampir seluruh

bentuk lain pada bab-bab berikutnya tetap memakainya sebagai dasar. Pembahasan mengikuti kerangka yang disusun oleh Richards dan Ford [18].

1.2 Aspek yang Dinilai pada Sebuah Arsitektur

Sebuah arsitektur tidak dinilai benar atau salah, melainkan sesuai atau tidak sesuai dengan kebutuhan. Penilaiannya memakai sejumlah aspek, dan aspek-aspek tersebut sering saling bertentangan. Tabel 1.1 merangkum lima aspek yang paling sering dipakai.

Tabel 1.1: Lima aspek yang paling sering dipakai untuk menilai arsitektur

Aspek	Pertanyaannya	Diukur dengan
Kemudahan perawatan	Berapa banyak berkas berubah untuk satu permintaan perubahan?	Jumlah berkas dan baris
Kemampuan skala	Apa yang terjadi ketika jumlah pengguna naik sepuluh kali?	Latensi pada beban bertingkat
Kinerja	Berapa lama satu permintaan dilayani?	Median dan persentil ke-95
Ketersediaan	Apa yang gagal ketika satu bagian mati?	Jumlah proses yang terhenti
Kemudahan pengujian	Berapa besar bagian yang dapat diuji tanpa infrastruktur?	Cakupan pengujian

Daftar aspek tersebut bukan satu-satunya yang dipakai. Microsoft menyusun daftar serupa untuk membandingkan gaya arsitektur pada panduan resminya [14].

Aspek-aspek tersebut jarang dapat dipenuhi seluruhnya sekaligus. Menaikkan kemampuan skala biasanya menurunkan kemudahan perawatan, sebab sistem harus dipecah menjadi lebih banyak bagian. Menaikkan kinerja sering menurunkan kemudahan pengujian, sebab jalan pintas mulai bermunculan. Pekerjaan seorang arsitek karena itu bukan mencari susunan terbaik, melainkan memilih aspek mana yang dikorbankan.

Setiap bab pada modul ini mengukur dua atau tiga aspek dari tabel tersebut, lalu melaporkan angkanya. Cara itu dipakai agar perbandingan antar bentuk arsitektur berdiri di atas angka, bukan di atas pendapat.

1.3 Studi Kasus Singkat

Tiga kejadian berikut menunjukkan bahwa keputusan arsitektural berakibat nyata, bukan sekadar soal kerapian kode.

Netflix. Pada 2008 layanan Netflix berhenti cukup lama akibat kerusakan

pada basis data tunggalnya. Seluruh sistem saat itu berupa satu kesatuan, dan kegagalan satu bagian menghentikan seluruhnya. Sesudah kejadian tersebut sistem dipecah menjadi banyak layanan terpisah, sehingga kegagalan satu layanan tidak lagi menghentikan yang lain.

Healthcare.gov. Pada peluncurannya tahun 2013 situs pendaftaran asuransi kesehatan Amerika Serikat tersebut nyaris tidak dapat dipakai. Salah satu sebabnya, sistem memanggil banyak sistem luar secara berurutan dan menunggu seluruh jawabannya sebelum menampilkan halaman. Satu sistem luar yang lambat membuat seluruh halaman ikut lambat.

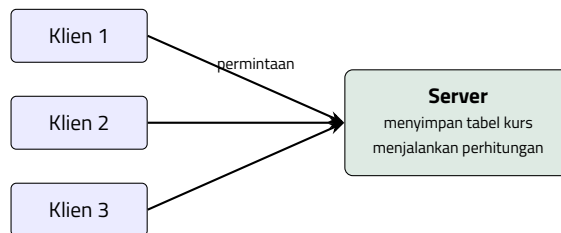
Sistem perbankan. Sistem yang memproses transaksi secara berkelompok pada akhir hari menunda pembaruan saldo sampai kelompok tersebut selesai diproses. Perpindahan ke pemrosesan berbasis event membuat saldo diperbarui segera setelah transaksi terjadi. Bab 9 membahas bentuk tersebut secara khusus.

Ketiganya memperlihatkan pola yang sama. Yang gagal bukan algoritmanya, melainkan susunan bagian-bagiannya beserta cara mereka saling menunggu.

1.4 Arsitektur Client-Server

Client-server membagi sistem menjadi dua peran. Klien meminta, server melayani. Gambar 1.1 menunjukkan susunannya. Banyak klien memanggil satu server, dan seluruh panah berawal dari klien.

Pemisahan peminta dan pelayan tersebut menjadi dasar hampir seluruh gaya arsitektur berikutnya. Fielding [4] menelusuri pembatasan yang melekat padanya, termasuk keharusan setiap permintaan menempuh jaringan sehingga biayanya tidak dapat dihapus.



Gambar 1.1: Banyak klien memanggil satu server. Seluruh permintaan berawal dari klien, dan server tidak pernah memanggil klien

Pembagian tersebut menentukan siapa menyimpan apa. Server menyimpan data bersama dan menjalankan aturan bisnis. Klien menyimpan keadaan tampilan saja. Karena tabel kurs hanya ada di server, memperbaruinya cukup dilakukan di satu tempat, dan seluruh klien langsung memakai nilai baru tersebut.

Analogi yang mendekati adalah warung dengan satu kasir. Pembeli antri dan memesan, kasir yang menghitung dan menyimpan uang. Pembeli tidak perlu tahu harga pokok setiap barang, dan perubahan harga cukup diberitahukan kepada kasir. Kelemahannya juga sama, yaitu ketika pembeli banyak, antrian memanjang walaupun setiap pesanan sebenarnya cepat dilayani.

Listing 1.1 memperlihatkan inti server pada contoh bab ini. Server membaca parameter permintaan, menghitung, lalu membalas dalam bentuk JavaScript Object Notation (JSON).

```

1 def do_GET(self):
2     """Membaca parameter permintaan, menghitung, lalu membalas dalam JSON."""
3     parameter = parse_qs(urlparse(self.path).query)
4     asal = parameter.get("asal", ["USD"])[0]
5     tujuan = parameter.get("tujuan", ["IDR"])[0]
6     nominal = float(parameter.get("nominal", ["1"])[0])
7
8     hasil = hitung_konversi(asal, tujuan, nominal)
9     if hasil is None:
10         self.kirim(404, {"galat": "Kurs tidak tersedia"})
11     else:
12         self.kirim(200, {"asal": asal, "tujuan": tujuan, "hasil": hasil})

```

Listing 1.1: Penanganan satu permintaan pada server, tersedia di `sources/chapter01/server.py`

Klien pada contoh tersebut sangat tipis. Klien hanya menyusun alamat, mengirim permintaan, lalu menampilkan jawaban. Tidak ada tabel kurs dan tidak ada perhitungan di sisi klien.

1.5 Dua Biaya yang Melekat

Pemisahan klien dan server menimbulkan dua biaya yang tidak dapat dihindari. Keduanya menjadi bahan pengukuran pada latihan bab ini.

Biaya perjalanan. Setiap permintaan harus menempuh jaringan dua kali, yaitu pergi dan pulang. Pada server di komputer yang sama biaya ini kecil, tetapi tetap ada. Pada server yang berada di kota lain, biaya ini menjadi bagian terbesar dari waktu tanggap, dan tidak dapat diperkecil dengan mempercepat perhitungannya.

Biaya antrian. Satu server melayani banyak klien. Ketika permintaan datang lebih cepat daripada kemampuan server melayaninya, sisanya menunggu. Server pada contoh bab ini melayani satu permintaan pada satu waktu, sehingga gejala tersebut muncul jelas begitu jumlah klien bertambah.

Tabel 1.2 merangkum hasil pengukuran nyata pada contoh bab ini. Kolom terakhir memperlihatkan gejala antrian tersebut.

Angka pada kolom terakhir melonjak tajam mulai delapan klien. Nilai median hanya naik dari 0,24 menjadi 1,48 milidetik, sedangkan nilai terbesarnya men-

Tabel 1.2: Latensi terhadap jumlah klien serentak pada server contoh

Jumlah klien serentak	Median (ms)	Terbesar (ms)
1	0,240	8,027
2	0,363	0,848
4	0,777	1,748
8	1,472	1011,138
16	1,481	1019,625

capai seribu milidetik. Sebagian permintaan karena itu menunggu sangat lama, sementara sebagian besar lainnya tetap cepat. Gejala tersebut tidak terlihat bila yang dilaporkan hanya nilai rata-rata, dan itulah alasan pengukuran pada modul ini selalu menyertakan nilai terbesar atau persentil ke-95.

1.6 Kelebihan dan Kekurangan

Tabel 1.3 merangkum untung rugi bentuk client-server.

Tabel 1.3: Kelebihan dan kekurangan arsitektur client-server

Kelebihan	Kekurangan
Data bersama disimpan di satu tempat, sehingga pembaruan cukup sekali	Server menjadi satu-satunya titik yang bila mati menghentikan seluruh layanan
Aturan bisnis tidak dapat diubah dari sisi klien	Setiap permintaan menanggung biaya perjalanan jaringan
Klien menjadi ringan dan mudah diganti	Server menjadi antrean bersama ketika klien bertambah
Perangkat klien tidak perlu bertenaga besar	Menambah kemampuan server lebih mahal daripada menambah klien

1.7 Ringkasan

Arsitektur adalah susunan bagian besar sistem beserta aturan hubungan di antaranya. Yang membedakannya dari rancangan rinci adalah biaya perubahan. Keputusan yang murah diubah bukan keputusan arsitektural, sedangkan keputusan yang menyentuh hampir seluruh berkas termasuk di dalamnya.

Sebuah arsitektur dinilai memakai beberapa aspek yang saling bertentangan, yaitu kemudahan perawatan, kemampuan skala, kinerja, ketersediaan, dan kemudahan pengujian. Aspek-aspek tersebut jarang dapat dipenuhi seluruhnya sekaligus, sehingga pekerjaan seorang arsitek adalah memilih aspek mana yang dikorbankan.

Ketiga studi kasus pada bab ini memperlihatkan pola yang sama. Yang gagal

bukan algoritmanya, melainkan susunan bagian-bagiannya beserta cara mereka saling menunggu. Perbaikannya pun bukan mempercepat perhitungan, melainkan mengubah susunannya.

Client-server membagi sistem menjadi peminta dan pelayan. Server menyimpan data bersama dan menjalankan aturan bisnis, sedangkan klien menyimpan keadaan tampilan saja. Pembagian tersebut membuat pembaruan data cukup dilakukan di satu tempat, dan membuat aturan bisnis tidak dapat diubah dari sisi klien.

Pembagian itu menimbulkan dua biaya tetap, yaitu perjalanan jaringan dan antrean di server. Pengukuran pada bab ini menunjukkan biaya antrean tidak terlihat pada nilai median, tetapi tampak jelas pada nilai terbesar. Karena itu setiap pengukuran pada modul ini melaporkan sebaran, bukan satu angka rata-rata.

1.8 Latihan

Ketiga latihan berikut memakai satu basis kode yang sama di `sources/chapter01/`, tetapi menysasar tiga masalah yang berbeda. Latihan 1 memeriksa pemisahan tanggung jawab. Latihan 2 mengukur biaya perjalanan. Latihan 3 mengukur perilaku server ketika klien bertambah. Seluruh pengukuran hanya dijalankan terhadap server milik sendiri, tidak terhadap sistem milik pihak lain. Hasil setiap mahasiswa dapat berbeda, dan yang dilaporkan adalah pola perbandingannya.

Lingkungan kerja disiapkan sekali saja, sebelum latihan pertama dijalankan. Skrip pada Listing 1.2 membuat *virtual environment* (venv) di `.venv`, lalu memasang pustaka yang tercantum di `sources/requirements.txt`. Venv memisahkan pustaka latihan dari Python bawaan sistem, sehingga percobaan pada satu bab tidak mengganggu bab lainnya. Perintahnya dijalankan dari akar repositori. Pada terminal baru, cukup baris pengaktifannya yang diulang, sedangkan pembuatannya tidak perlu diulang. Bagi pengguna Windows, baris pengaktifan tersebut diganti dengan `.venv\Scripts\activate`.

Listing 1.2: Menyiapkan venv sekali sebelum seluruh latihan, skrip tersedia di `sources/siapkan_venv.sh`

```
1 bash sources/siapkan_venv.sh
2 source .venv/bin/activate
3 python --version
4 # Keluarannya: Python 3.14.7
```

Server dijalankan lebih dahulu pada satu terminal seperti pada Listing 1.3, lalu skrip lainnya pada terminal kedua. Venv diaktifkan pada kedua terminal tersebut.

Listing 1.3: Menjalankan server pada terminal pertama

```

1 cd sources/chapter01
2 python server.py

```

Latihan 1: Memetakan Tanggung Jawab

Latihan ini membuktikan bahwa klien tidak menyimpan maupun menghitung apa pun. Langkah kerjanya sebagai berikut.

1. Jalankan `python klien.py USD IDR 100` pada terminal kedua, lalu catat jawabannya.
2. Buka `sources/chapter01/klien.py`, lalu cari tabel kurs di dalamnya.
3. Ubah satu nilai pada tabel kurs di `sources/chapter01/server.py`, jalankan ulang server, lalu jalankan klien tanpa mengubah apa pun.
4. Matikan server, lalu jalankan klien sekali lagi dan catat yang terjadi.

Tabel 1.4 memperlihatkan bentuk pengisian yang diharapkan.

Tabel 1.4: Contoh hasil Latihan 1 yang sudah terisi

Pertanyaan	Jawaban	Bukti
Klien menyimpan tabel kurs	tidak	Tidak ada kamus KURS di <code>klien.py</code>
Klien ikut berubah setelah kurs diubah	tidak	Jawaban berubah tanpa menyunting klien

Isikan hasil pengamatan pada Tabel 1.5.

Tabel 1.5: Lembar isian Latihan 1

Pertanyaan	Jawaban	Bukti
Klien menyimpan tabel kurs	...	...
Klien menghitung konversi	...	...
Klien ikut berubah setelah kurs diubah	...	...
Yang terjadi ketika server dimatikan	...	...

Jawab dua pertanyaan berikut. Pertama, jelaskan keuntungan menyimpan tabel kurs hanya di server. Kedua, jelaskan kerugiannya berdasarkan yang terjadi ketika server dimatikan.

Latihan 2: Mengukur Biaya Perjalanan

Latihan ini mengukur waktu bolak-balik satu permintaan. Langkah kerjanya sebagai berikut.

1. Jalankan `python ukur_latensi.py`, lalu catat kelima angka yang dilaporkan.

2. Ulangi pengukuran sebanyak tiga kali.
3. Bandingkan nilai terkecil dengan nilai terbesar pada setiap pengukuran.
4. Tambahkan perhitungan berulang di dalam `hitung_konversi` agar server bekerja lebih lama, lalu ukur kembali.

Tabel 1.6 memperlihatkan hasil satu kali pengukuran nyata.

Tabel 1.6: Contoh hasil Latihan 2 yang sudah terisi

Pengukuran	Median	Persentil ke-95	Terbesar
Ke-1, 200 permintaan	0,237 ms	0,351 ms	7,391 ms

Isikan hasil pengukuran pada Tabel 1.7.

Tabel 1.7: Lembar isian Latihan 2

Pengukuran	Median	Persentil ke-95	Terbesar
Ke-1	...	...	...
Ke-2	...	...	...
Ke-3	...	...	...
Setelah server dibuat lebih lambat	...	...	...

Jawab dua pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, nilai terbesar pada pengukuran pertama jauh melebihi mediannya, sehingga jelaskan sebabnya. Kedua, jelaskan bagian mana dari waktu tersebut yang tetap ada walaupun perhitungan di server dipercepat.

Latihan 3: Menambah Jumlah Klien

Latihan ini membuktikan bahwa server adalah antrean bersama. Langkah kerjanya sebagai berikut.

1. Jalankan `python ukur_beban.py`, lalu catat median dan nilai terbesar pada setiap jumlah klien.
2. Tandai jumlah klien saat nilai terbesar mulai melonjak.
3. Bandingkan pertumbuhan median terhadap pertumbuhan nilai terbesar.
4. Jelaskan mengapa keduanya tumbuh dengan pola yang berbeda.

Tabel 1.8 memperlihatkan hasil satu kali pengukuran nyata.

Tabel 1.8: Contoh hasil Latihan 3 yang sudah terisi

Jumlah klien	Median (ms)	Terbesar (ms)
4	0,777	1,748
8	1,472	1011,138

Isikan hasil pengukuran pada Tabel 1.9.

Tabel 1.9: Lembar isian Latihan 3

Jumlah klien	Median (ms)	Terbesar (ms)
1	...	...
2	...	...
4	...	...
8	...	...
16	...	...
Jumlah klien saat lonjakan mulai	...	

Jawab tiga pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, median tumbuh perlahan sedangkan nilai terbesar melonjak, sehingga jelaskan apa yang dialami sebagian kecil permintaan tersebut. Kedua, jelaskan mengapa gejala itu hilang bila hanya nilai rata-rata yang dilaporkan. Ketiga, dengan menggabungkan hasil ketiga latihan, sebutkan aspek dari Tabel 1.1 yang paling tertekan pada arsitektur client-server beserta angka yang menjadi dasarnya.

Bab 2

Layered Architecture

Tujuan Pembelajaran

1. Menjelaskan pembagian tanggung jawab keempat *layer* pada *layered architecture* beserta arah ketergantungannya.
2. Membedakan *closed layer* dari *open layer*, lalu menentukan kapan masing-masing bentuk layak dipilih.
3. Mengukur kedalaman *call stack*, *layer violation* ketergantungan, dan selisih waktu antara kedua bentuk pada satu aplikasi yang sama.

2.1 Pendahuluan

Layered architecture menata sebuah aplikasi menjadi beberapa *layer* yang bertumpuk. Setiap *layer* mengerjakan satu jenis urusan teknis, lalu memanggil *layer* di bawahnya bila membutuhkan sesuatu. Bentuk ini juga dikenal sebagai arsitektur *n-tier* [13]. Bentuk ini menjadi pilihan pertama bagi sebagian besar sistem, dan sering dipakai tanpa pertimbangan panjang ketika tim belum memutuskan arsitektur lain.

Fowler [6] mencatat bahwa pembagian menjadi tampilan, domain, dan data merupakan pembagian yang paling sering dipakai, sekaligus yang paling sering dipakai tanpa alasan yang dipikirkan.

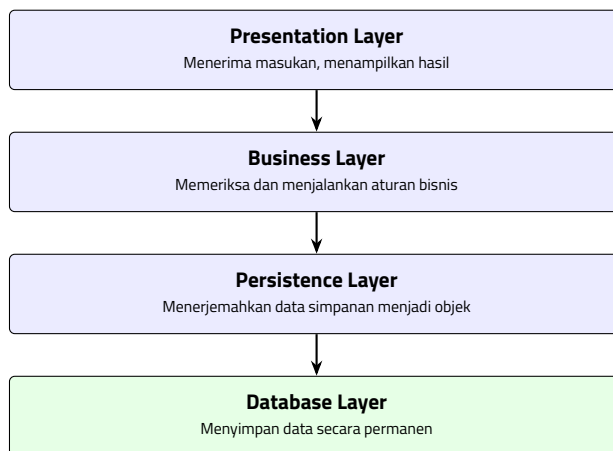
Salah satu alasan pemakaiannya bersifat organisasi, bukan teknis. Pembagian *layer* mengikuti pembagian kerja yang sudah ada di banyak tim pengembang. Ada yang mengerjakan antarmuka, ada yang mengerjakan aturan bisnis, dan ada yang mengerjakan basis data. Struktur tim tersebut memetakan diri dengan rapi ke struktur *layer*, sehingga setiap orang tahu berkas mana yang menjadi tanggung

jawabnya.

Kemudahan itu ada harganya. Ketika aplikasi membesar, pemisahan yang tadinya menolong justru memperlambat perubahan. Satu fitur baru sering menyentuh keempat *layer* sekaligus. Bab ini membahas pembagian *layer*, aturan ketergantungan di antaranya, serta pilihan antara *closed layer* dan *open layer*. Pembahasan mengikuti kerangka yang disusun oleh Richards dan Ford [18].

2.2 Pembagian *Layer*

Bentuk baku *layered architecture* terdiri atas empat *layer*. Gambar 2.1 menunjukkan susunannya beserta arah panggilan yang diizinkan. Panah hanya menunjuk ke bawah, dan tidak ada panah yang melompati satu *layer*.



Gambar 2.1: Empat *layer* baku beserta arah panggilan yang diizinkan

Presentation layer menangani segala hal yang dilihat pengguna. *Layer* ini menerima masukan, memanggil *business layer*, lalu menampilkan hasilnya. *Layer* ini tidak mengetahui asal data maupun cara data disimpan. Contohnya menyusun teks 100 USD = 1.625.000 IDR lengkap dengan pemisah ribuan, menampilkan pesan galat ketika nominal ditolak, dan mengubah daftar kurs menjadi baris-baris teks. Bila layar diganti menjadi API, seluruh isi *layer* ini dibuang dan ditulis ulang.

Business layer memuat aturan yang membuat sistem tersebut bernilai. Pemeriksaan kelayakan, perhitungan, dan keputusan proses berada di sini. *Layer* ini tidak mengetahui bentuk antarmuka maupun bentuk tabel basis data. Contohnya menolak nominal yang bernilai nol atau negatif, menolak nominal yang melewati batas wajar, memeriksa token pemanggil, dan memeriksa apakah peran pemanggil berhak membaca kurs jual. Seluruh aturan tersebut tetap berlaku walaupun layarnya diganti.

Persistence layer menjembatani objek di dalam program dengan baris di dalam basis data. Kueri Structured Query Language (SQL) ditulis di *layer* ini, sehingga *business layer* tidak perlu mengetahui nama tabel. Contohnya menjalankan kueri pencarian satu kurs, mengubah baris hasil kueri menjadi tupel siap pakai, dan mengembalikan nilai kosong ketika pasangan mata uang tidak ditemukan. Bila basis datanya berganti, hanya isi *layer* inilah yang ditulis ulang.

Database layer menyimpan data secara permanen. *Layer* ini biasanya berupa perangkat lunak tersendiri, bukan kode yang ditulis tim pengembang. Contohnya tabel kurs beserta kolom nilai dan margin, kunci utama berupa pasangan kode mata uang, serta aturan bahwa satu pasangan hanya boleh muncul satu kali. Kolom `margin` memuat margin keuntungan bank. Keputusan menyimpannya pada kolom terpisah diambil di *layer* ini, dan keputusan itulah yang memungkinkan kurs dasar dibaca tanpa ikut membuka keuntungannya.

Pembagian tersebut memberi satu keuntungan yang jelas. Perubahan pada satu urusan teknis tertahan di satu *layer*. Penggantian basis data, misalnya, cukup menyentuh *persistence layer* selama bentuk objek yang dikembalikannya tetap. Tabel 2.1 merangkum tanggung jawab dan larangan setiap *layer*.

Tabel 2.1: Tanggung jawab dan larangan setiap *layer*

<i>Layer</i>	Tanggung jawab	Tidak boleh mengetahui
Presentation	Menerima masukan dan menampilkan hasil	Nama tabel, kueri, sumber data
Business	Memeriksa aturan dan menghitung hasil	Bentuk antarmuka, bentuk tabel
Persistence	Menerjemahkan baris menjadi objek	Aturan bisnis, kebutuhan tampilan
Database	Menyimpan data secara permanen	Seluruh <i>layer</i> di atasnya

Agar pembagian tersebut tidak berhenti sebagai daftar, Tabel 2.2 menunjukkan isi setiap *layer* pada aplikasi kurs yang dipakai sepanjang bab ini. Perhatikan bahwa satu kalimat kebutuhan biasanya jatuh ke satu *layer* saja.

Tabel 2.2: Contoh isi setiap *layer* pada aplikasi kurs

<i>Layer</i>	Contoh isinya	Kalimat kebutuhan
Presentation	Menyusun teks 100 USD = 1.625.000 IDR	Hasil ditampilkan dengan pemisah ribuan
Business	Menolak nominal nol, memeriksa token dan peran	Hanya staf boleh melihat kurs jual
Persistence	Mengubah baris tabel menjadi tupel	Kurs disimpan per pasangan mata uang
Database	Tabel kurs beserta kolom <code>margin</code>	Margin keuntungan disimpan terpisah dari nilai dasar

Uji cepat untuk menempatkan sebuah kebutuhan cukup satu pertanyaan. Bila

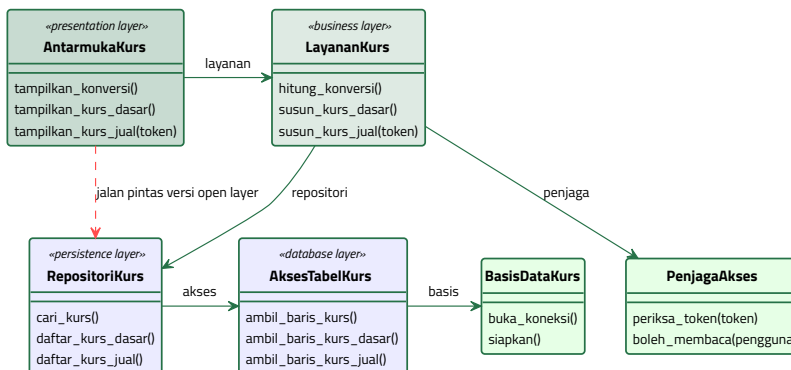
kebutuhan tersebut tetap berlaku walaupun aplikasi berganti dari layar menjadi API, kebutuhan itu milik *business layer*. Bila kebutuhan tersebut hilang ketika layarnya diganti, kebutuhan itu milik *presentation layer*.

2.3 Empat *Layer* sebagai Empat Kelas

Pembagian pada Tabel 2.2 masih berupa daftar. Agar pembagian tersebut terbaca di dalam kode, setiap *layer* diwujudkan menjadi satu kelas. Kelas *layer* atas menyimpan rujukan ke kelas *layer* tepat di bawahnya, dan rujukan itu diterima lewat konstruktor. Arah ketergantungan karena itu terbaca dari tanda tangan konstrukturnya, tanpa perlu menjalankan satu baris pun kode.

Susunan tersebut menyerupai kepanitiaan di kampus. Ketua panitia hanya memegang nomor kontak koordinator seksi, sedangkan koordinator seksi memegang nomor kontak anggotanya. Siapa memegang nomor siapa menentukan siapa yang dapat menghubungi siapa, sehingga jalur komunikasinya sudah tertentu sejak kepanitiaan dibentuk.

Gambar 2.2 memperlihatkan keempat kelas tersebut beserta dua kolaborator di luar susunan *layer*. Panah menunjuk kelas yang dipegang, dan keterangan pada panah menyebutkan nama atributnya.



Gambar 2.2: Empat kelas *layer* beserta dua kolaboratnya, dengan jalan pintas versi *open* sebagai garis putus-putus

Dua kotak pada gambar tersebut bukan *layer*. PenjagaAkses menegakkan aturan tentang siapa yang boleh membaca kurs jual, sehingga kedudukannya sebagai kolaborator *business layer*. BasisDataKurs mewakili berkas SQLite beserta skemanya, yaitu perangkat lunak penyimpanan yang dipakai *layer* basis data.

Perangkaian keempat kelas dikumpulkan pada satu fungsi, seperti pada Listing 2.1. Tidak ada kelas yang membuat kolaboratnya sendiri, sehingga penguji-

an dapat menyisipkan pengganti tanpa menyentuh isi kelas mana pun.

```

1 def rakit_antarmuka():
2     akses = AksesTabelKurs(BasisDataKurs())
3     repositori = RepositoriKurs(akses)
4     layanan = LayananKurs(repositori, PenjagaAkses())
5     return AntarmukaKurs(layanan)

```

Listing 2.1: Perangkatian keempat lapisan, tersedia di `sources/chapter02/aplikasi_tertutup.py`

2.4 Aturan Ketergantungan

Aturan pokok *layered architecture* hanya satu. Sebuah *layer* boleh memanggil *layer* tepat di bawahnya, dan tidak boleh memanggil ke atas. Aturan itu terdengar sederhana, tetapi menentukan hampir seluruh sifat arsitektur ini.

Analogi yang mendekati adalah loket pembayaran uang kuliah di kampus. Mahasiswa berbicara kepada petugas loket, petugas loket berbicara kepada bagian keuangan, lalu bagian keuangan berbicara kepada bank. Mahasiswa tidak menelepon bank secara langsung, dan bank tidak memanggil mahasiswa. Setiap pihak hanya mengenal tetangga terdekatnya.

Aturan tersebut punya dua konsekuensi praktis. Pertama, satu *layer* dapat diganti tanpa menyentuh *layer* lain, selama bentuk panggilannya tetap sama. Jadi, tidak perlu mengubah *layer* lain. Kedua, satu permintaan yang sederhana tetap harus melewati seluruh *layer*, walaupun sebagian *layer* tidak mengerjakan apa pun.

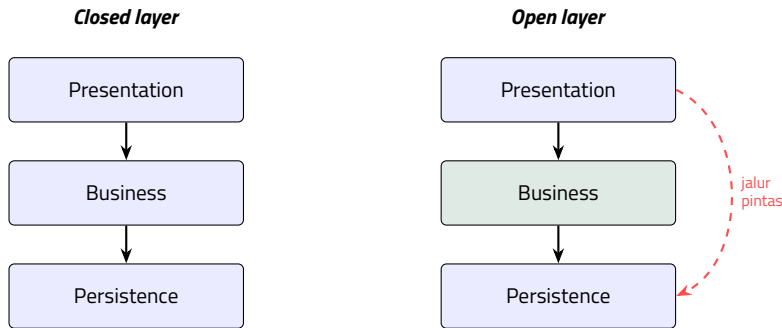
Konsekuensi kedua itulah yang menimbulkan persoalan berikutnya. Pertanyaannya, apakah setiap permintaan benar-benar wajib melewati semua *layer*.

2.5 Closed Layer dan Open Layer

Dua jawaban atas pertanyaan tersebut menghasilkan dua bentuk yang berbeda. Gambar 2.3 menunjukkan keduanya berdampingan.

Pada **closed layer**, setiap panggilan wajib melewati seluruh *layer* tanpa pengecualian. Susunan ini menjaga arah ketergantungan tetap bersih. Harganya muncul pada permintaan yang tidak punya aturan bisnis sama sekali. Kelas *business layer* lalu hanya meneruskan panggilan tanpa menambahkan apa pun.

Keadaan tersebut dikenal sebagai *architecture sinkhole anti-pattern*, yaitu permintaan yang mengalir melewati beberapa *layer* tanpa satu pun *layer* mengerjakan sesuatu. Permintaan hanya turun untuk mengambil data, lalu naik kembali tanpa diolah [18]. Keadaan ini hampir tidak mungkin dihapus sepenuhnya, sehingga yang diusahakan adalah menekan jumlahnya, bukan menghilangkannya. Yang perlu



Gambar 2.3: *Closed layer* mewajibkan setiap panggilan melewati *business layer*, sedangkan *open layer* mengizinkan jalur pintas

diperiksa adalah apakah *business layer* lebih sering meneruskan panggilan daripada mengerjakan sesuatu. Analogi yang mendekati adalah surat pengantar yang harus melewati tiga meja. Meja pertama membaca isinya dan memutuskan, sedangkan meja kedua dan ketiga hanya membubuhkan cap lalu meneruskan. Ketiga meja tersebut sama-sama menambah waktu, tetapi hanya satu yang menambah nilai. Bila keadaan itu dibiarkan, prosedurnya terlihat rapi di atas kertas namun tidak berguna dalam praktik.

Pada ***open layer***, permintaan tanpa aturan bisnis boleh memangkas satu lompatan. *Presentation layer* memanggil *persistence layer* secara langsung. Bentuk ini memangkas kode *pass-through*, tetapi menimbulkan ketergantungan baru.

2.6 Kapan Jalan Pintas Dapat Dibenarkan

Pertanyaan yang sebenarnya bukan apakah jalan pintas lebih cepat, melainkan apakah ada tuntutan bisnis yang ikut dilewati. Contoh pada bab ini memakai dua permintaan yang membaca tabel yang sama, tetapi tuntutan bisnisnya berbeda.

Kurs dasar adalah nilai tukar yang diterbitkan untuk umum. Bank Indonesia menerbitkan angka semacam itu setiap hari kerja dengan nama Jakarta Interbank Spot Dollar Rate (JISDOR), dan siapa pun boleh membacanya [1]. Tidak ada tuntutan bisnis yang berlaku pada pembacaan tersebut.

Kurs jual adalah kurs dasar ditambah margin keuntungan bank, yaitu harga yang dibayar nasabah ketika membeli valuta asing. Besar keuntungan itu informasi komersial, dan tuntutan bisnisnya menyatakan hanya staf yang boleh membacanya. Pemeriksaan siapa pemanggilnya dan apakah pemanggil tersebut berhak karena itu wajib dijalankan.

Kedua permintaan tersebut membaca tabel yang sama, sehingga tabel bukan

yang menentukan. Yang menentukan adalah tuntutan bisnis yang melekat pada masing-masing permintaan. Tabel 2.3 merangkum perbedaannya.

Tabel 2.3: Dua permintaan pada tabel yang sama dengan tuntutan bisnis berbeda

Permintaan	Tuntutan bisnis	Jalan pintas
Kurs dasar	Tidak ada, data memang untuk umum	Dapat dibenarkan
Kurs jual	Autentikasi dan otorisasi, sebab memuat margin keuntungan	Tidak dapat dibenarkan

Listing 2.2 memperlihatkan metode *pass-through* pada kurs dasar. Metode tersebut memang tidak menambahkan apa pun, dan itu wajar.

```

1 def susun_kurs_dasar(self):
2     """Meneruskan panggilan tanpa menambahkan apa pun.
3
4     Kurs dasar memang diterbitkan untuk umum, meniru JISDOR yang diumumkan
5     Bank Indonesia setiap hari kerja. Tidak ada tuntutan bisnis yang berlaku,
6     sehingga metode ini murni pass-through.
7     """
8     return self.repositori.daftar_kurs_dasar()
```

Listing 2.2: Metode *pass-through* yang wajar pada kelas `LayananKurs`, tersedia di `sources/chapter02/aplikasi_tertutup.py`

Bandingkan dengan Listing 2.3. Metode kedua ini menjalankan dua pemeriksaan sebelum data diberikan, dan kedua pemeriksaan itulah yang hilang bila *business layer* dilewati.

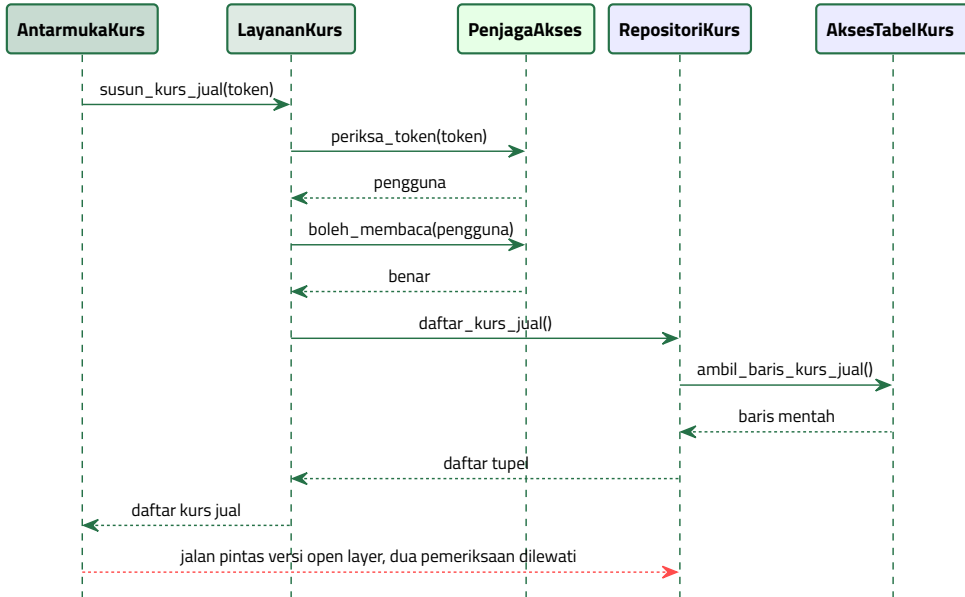
```

1 def susun_kurs_jual(self, token):
2     """Memeriksa autentikasi dan otorisasi sebelum kurs jual diberikan.
3
4     Kurs jual adalah kurs dasar ditambah margin keuntungan bank. Besar
5     keuntungan itu informasi komersial, sehingga tuntutan bisnisnya menyatakan
6     hanya staf yang boleh membacanya. Tuntutan itulah yang ikut hilang bila
7     lapisan bisnis dilewati.
8     """
9     pengguna = self.penjaga.periksa_token(token)
10    if pengguna is None:
11        raise PermissionError("Token tidak dikenali")
12    if not self.penjaga.boleh_membaca(pengguna):
13        raise PermissionError(f"Peran {pengguna['peran']} tidak berhak membaca")
14    return self.repositori.daftar_kurs_jual()
```

Listing 2.3: Pemeriksaan yang dituntut lapisan bisnis, tersedia di `sources/chapter02/aplikasi_tertutup.py`

Pada versi *open*, kelas `AntarmukaKurs` memanggil `RepositoriKurs` secara langsung. Pemanggil tidak perlu menyerahkan token, dan margin keuntungan ikut terbaca siapa pun. Jalan pintas tersebut memang lebih cepat, sebab dua pemeriksaan tidak dijalankan. Kecepatan itu dibeli dengan melepas tuntutan bisnis.

Gambar 2.4 memperlihatkan urutan panggilan pada permintaan kurs jual. Sepuluh panah pertama adalah jalur *closed*, dan dua di antaranya adalah pemeriksaan yang dituntut *business layer*. Panah putus-putus paling bawah adalah jalur *open*.



Gambar 2.4: Urutan panggilan permintaan kurs jual pada jalur *closed*, dengan jalur *open* sebagai panah putus-putus paling bawah

Perhatikan letak kedua pemeriksaan tersebut. Keduanya berada di antara panah pertama dan panggilan ke *persistence layer*. Jalur *open* memotong tepat di bagian itu, sehingga yang hilang bukan sekadar satu lompatan, melainkan seluruh tuntutan bisnis yang menempel pada permintaan tersebut.

Kasus berikut memperlihatkan akibatnya. Sebuah tim membuat halaman perbandingan kurs yang terbuka untuk pengunjung, tanpa perlu masuk lebih dahulu. Agar cepat selesai, halaman itu langsung membaca *persistence layer*. Waktu itu pilihannya masih aman, sebab yang tampil hanya kurs dasar. Beberapa bulan berikutnya, kurs jual diminta ikut tampil di halaman yang sama. Nama metodenya tinggal ditukar, sebab jalannya sudah terbuka sejak awal. Sejak saat itu margin keuntungan bank ikut terbaca siapa saja. Tidak ada pesan galat dan tidak ada yang gagal, sehingga tidak ada yang curiga.

Pola seperti itu sering berulang. Jalan pintas dibuat untuk data yang memang publik, lalu belakangan dipakai ulang untuk data yang tidak publik. Karena itu setiap jalan pintas sebaiknya dicatat beserta alasannya, agar pemakaian berikutnya diperiksa ulang, bukan diteruskan begitu saja.

Satu hal lain sering terlupakan. *Layer* mana yang tertutup dan *layer* mana yang terbuka wajib dicatat dan disepakati satu tim. Tanpa catatan tersebut, setiap orang menebak sendiri aturan yang berlaku, dan sistem berakhir dengan ketergantungan yang rapat serta sulit diuji.

2.7 Kelebihan dan Kekurangan

Tabel 2.4 merangkum untung rugi kedua bentuk tersebut. Perbandingan ini menjadi bahan pertimbangan sebelum memilih salah satunya.

Tabel 2.4: Perbandingan *closed layer* dan *open layer*

Aspek	<i>Closed layer</i>	<i>Open layer</i>
Arah ketergantungan	Bersih, hanya ke <i>layer</i> tepat di bawah	Presentation ikut mengenal persistensi
Jumlah kode	Bertambah oleh metode <i>pass-through</i>	Lebih sedikit
Kecepatan	Satu lompatan lebih banyak per permintaan	Satu lompatan lebih sedikit
Dampak perubahan	Tertahan di satu <i>layer</i>	Dapat merambat ke <i>presentation layer</i>
Cocok untuk	Sistem dengan aturan bisnis padat	Pembacaan sederhana tanpa aturan bisnis

Kelebihan utama *layered architecture* secara umum ada tiga. Pembagian kerja menjadi jelas, pengujian setiap *layer* dapat berdiri sendiri, dan penggantian satu teknologi tidak menyentuh seluruh sistem. Ketiganya membuat bentuk ini cocok untuk sistem berukuran kecil sampai menengah.

Kekurangannya muncul pada sistem besar. Satu fitur baru sering menyentuh keempat *layer* sekaligus, sehingga tidak ada satu bagian pun yang dapat di-*deploy* sendiri. Selain itu seluruh *layer* tumbuh bersama-sama, sehingga sistem tetap berupa satu kesatuan yang harus dirilis sekaligus.

2.8 Ringkasan

Layered architecture membagi aplikasi menjadi empat *layer*, yaitu presentasi, bisnis, persistensi, dan basis data. Setiap *layer* mengerjakan satu jenis urusan teknis, lalu memanggil *layer* di bawahnya. Pembagian tersebut mengikuti pembagian kerja yang sudah ada di banyak tim pengembang, dan itulah salah satu alasan bentuk ini banyak dipakai.

Aturan pokoknya hanya satu, yaitu panggilan hanya boleh mengarah ke bawah. Aturan itu menjaga agar perubahan pada satu urusan teknis tertahan di satu *layer*.

Penggantian basis data cukup menyentuh *persistence layer*, selama bentuk objek yang dikembalikannya tidak berubah.

Aturan yang sama menimbulkan biaya pada permintaan sederhana. Permintaan yang tidak punya aturan bisnis tetap harus melewati *business layer*, sehingga muncul metode yang hanya meneruskan panggilan. Bila metode *pass-through* sudah lebih banyak daripada metode yang benar-benar bekerja, susunan lapisannya perlu ditinjau.

Open layer mengizinkan jalur pintas untuk permintaan tanpa aturan bisnis. Pengukuran seratus putaran pada latihan bab ini mencatat selisih median 0,0121 milidetik, sedangkan jalur pintas hanya tercatat lebih cepat pada 48 putaran. Selisih sekecil itu tenggelam oleh *range* di dalam satu jalur, yang mencapai 0,7754 milidetik atau 64 kali lipatnya.

Perbandingan itulah yang menentukan. Penghematannya kecil dan sulit dipas-tikan, sedangkan dua pemeriksaan yang dilepas hilang pada seluruh permintaan tanpa perkecualian. Kurs dasar memang diterbitkan untuk umum, sehingga jalan pintas padanya dapat dibenarkan. Kurs jual memuat margin keuntungan, dan tuntutan bisnisnya menyatakan hanya staf yang boleh membacanya, sehingga jalan pintas padanya tidak dapat dibenarkan. Keputusan diambil dengan membaca tuntutan bisnis setiap permintaan, bukan dengan mengikuti kebiasaan.

2.9 Latihan

Ketiga latihan berikut memakai satu basis kode yang sama di `sources/chapter02/`, tetapi menysasar tiga masalah yang berbeda. Latihan 1 menelusuri *call stack* dengan membaca kode, tanpa menjalankannya. Latihan 2 memeriksa arah ketergantungan yang dilanggar. Latihan 3 mengukur biaya waktu satu lompatan antar *layer*. Penilaian didasarkan pada kelengkapan tabel hasil dan ketepatan jawaban analisisnya, bukan pada kecepatan pengerjaan. Hasil pengukuran setiap mahasiswa akan berbeda, dan yang dilaporkan adalah pola perbandingannya, bukan angka persisnya.

Venv yang dibuat pada Bab 1 dipakai kembali, sehingga cukup diaktifkan seperti pada Listing 2.4. Bila `venv` tersebut belum pernah dibuat, baris pertama membuatnya lebih dahulu, lalu memasang pustaka yang tercantum di `sources/requirements.txt`. Skripnya aman dijalankan berulang, sebab `venv` yang sudah ada tidak dibuat ulang. Kedua perintah dijalankan dari akar repositori.

Listing 2.4: Mengaktifkan `venv`, skrip tersedia di `sources/siapkan_venv.sh`

```
1 bash sources/siapkan_venv.sh
2 source .venv/bin/activate
```

Sesudah venv aktif, seluruh latihan dijalankan seperti pada Listing 2.5.

Listing 2.5: Persiapan sebelum ketiga latihan

```
1 cd sources/chapter02
2 python basis_data.py
```

Latihan 1: Menelusuri Jalan Satu Permintaan

Latihan ini tidak menjalankan satu perintah pun. Yang dikerjakan adalah membaca kode, lalu mengikuti satu permintaan sejak titik masuknya sampai baris yang benar-benar membaca basis data. Baris itu adalah `koneksi.execute`.

Ada empat jalan yang perlu ditelusuri, yaitu dua permintaan pada dua versi aplikasi. Listing 2.6 menuliskan keempat jalan tersebut, tetapi hanya kedua ujungnya. Isi di antara kedua ujung itulah yang dicari.

Listing 2.6: Empat jalan yang ditelusuri pada Latihan 1, kedua berkasnya tersedia di `sources/chapter02/`

```
1 # closed, aplikasi_tertutup.py
2 antarmuka.tampilkan_kurs_jual(token) ----> koneksi.execute(KUERI_KURS_JUAL)
3 antarmuka.tampilkan_kurs_dasar() ----> koneksi.execute(KUERI_KURS_DASAR)
4
5 # open, aplikasi_terbuka.py
6 antarmuka.tampilkan_kurs_jual() ----> koneksi.execute(KUERI_KURS_JUAL)
7 antarmuka.tampilkan_kurs_dasar() ----> koneksi.execute(KUERI_KURS_DASAR)
```

Langkah kerjanya sebagai berikut.

1. Buka `sources/chapter02/aplikasi_tertutup.py`. Mulai dari `tampilkan_kurs_jual`, lalu catat setiap metode yang dipanggil sampai `koneksi.execute` tercapai.
2. Kerjakan hal yang sama untuk `tampilkan_kurs_dasar`.
3. Buka `sources/chapter02/aplikasi_terbuka.py`, lalu telusuri kedua permintaan yang sama di sana.
4. Hitung berapa metode yang dilewati pada masing-masing jalan.
5. Tandai metode yang tubuhnya hanya satu baris `return` berisi pemanggilan. Metode seperti itu disebut *pass-through*.
6. Tandai pula pemeriksaan yang dijalankan sebelum data dibaca.

Tabel 2.5 berisi hasil keempat penelusuran tersebut sebagai contoh. Kolom metode menghitung seluruh metode yang dipanggil sampai `koneksi.execute`, sedangkan pemeriksaan dihitung pada kolom tersendiri.

Isikan hasil penelusuran pada Tabel 2.6.

Tabel 2.5: Contoh hasil Latihan 1 yang sudah terisi

Permintaan	Berkas	Metode	Periksa	Pass
Kurs jual	aplikasi_tertutup.py	5	2	0
Kurs jual	aplikasi_terbuka.py	4	0	0
Kurs dasar	aplikasi_tertutup.py	5	0	1
Kurs dasar	aplikasi_terbuka.py	4	0	0

Tabel 2.6: Lembar isian Latihan 1

Permintaan	Berkas	Metode	Periksa	Pass
Kurs jual	aplikasi_tertutup.py	...	...	...
Kurs jual	aplikasi_terbuka.py	...	...	...
Kurs dasar	aplikasi_tertutup.py	...	...	...
Kurs dasar	aplikasi_terbuka.py	...	...	...
Nama metode <i>pass-through</i>	...			

Jawab tiga pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, versi *closed* selalu satu metode lebih panjang daripada versi *open*, sehingga sebutkan metode mana yang menambah panjang itu beserta isinya. Kedua, pada kurs dasar metode tersebut tidak membawa pemeriksaan apa pun, sedangkan pada kurs jual membawa dua, sehingga jelaskan mengapa akibat memangkasnya berbeda. Ketiga, hitung bagian metode yang benar-benar menambahkan sesuatu pada setiap jalan, lalu bandingkan keempat angkanya.

Latihan 2: Menemukan *Layer Violation*

Latihan ini membuktikan bahwa jalan pintas menimbulkan ketergantungan baru. *Violation*-nya dicari dengan membaca kode, bukan dengan menjalankan alat bantu. Aturannya hanya satu, yaitu sebuah kelas hanya boleh memegang kelas satu *layer* tepat di bawahnya. Urutannya *presentation*, *business*, *persistence*, lalu *database*. Rujukan yang melompati satu *layer* atau lebih adalah *violation*. Langkah kerjanya sebagai berikut.

1. Buka kedua berkas aplikasi, lalu salin isi konstanta LAPISAN. Konstanta itu menyebutkan *layer* setiap kelas.
2. Baca konstruktor setiap kelas, lalu catat kelas yang dipegangnya dari anotasi tipe pada parameternya. Tabel 2.7 menyebutkan urutan pemeriksaan yang paling cepat.
3. Gambarkan satu panah dari kelas pemegang ke kelas yang dipegang, lalu bandingkan arahnya dengan urutan keempat *layer* tersebut.
4. Tandai panah yang melompati satu *layer* atau lebih, lalu catat metode mana saja yang benar-benar memakai rujukan itu.

5. Jalankan kedua aplikasi seperti pada Listing 2.7, lalu bandingkan perlakuan-nya terhadap ketiga token contoh.

Listing 2.7 memperlihatkan akibat *violation* tersebut saat aplikasinya dijalankan. Versi *closed* memisahkan kurs dasar dari kurs jual menjadi dua perintah, lalu meminta token pada perintah kedua. Selisih kedua keluarannya, yaitu 90 rupiah pada EUR dan 75 rupiah pada USD, adalah margin keuntungan yang dijaga. Versi *open* menyerahkan angka yang sudah bermargin itu tanpa meminta token sama sekali.

Listing 2.7: Menjalankan kedua aplikasi, keduanya tersedia di `sources/chapter02/`

```

1 python aplikasi_tertutup.py dasar
2 # Keluarannya, dua baris pertama dari empat, tanpa token sama sekali:
3 # EUR -> IDR: 17600.0
4 # USD -> IDR: 16250.0
5
6 python aplikasi_tertutup.py jual token-andi
7 # Keluarannya, angka yang sama ditambah margin keuntungan:
8 # EUR -> IDR: 17690.0
9 # USD -> IDR: 16325.0
10
11 python aplikasi_tertutup.py jual token-budi
12 # ditolak: Peran tamu tidak berhak membaca
13
14 python aplikasi_tertutup.py jual token-palsu
15 # ditolak: Token tidak dikenali
16
17 python aplikasi_terbuka.py
18 # Keluarannya, kurs jual beserta marginnya tanpa satu pun token:
19 # dasar: EUR -> IDR: 17600.0
20 # jual : EUR -> IDR: 17690.0
21 # Tidak ada token yang diminta, dan tidak ada pemeriksaan yang dijalankan.
```

Ketiga token tersebut memisahkan dua pemeriksaan yang berbeda. Token milik Budi lolos autentikasi tetapi gagal otorisasi, sedangkan token palsu gagal sejak autentikasi. Keduanya sama-sama tidak pernah berjalan pada versi *open*, sehingga margin keuntungan di sana terbaca siapa saja.

Pencarian akan lebih cepat bila dimulai dari konstruktor, bukan dari tubuh metodenya. Anotasi tipe pada konstruktor menyebutkan kelas yang dipegang, sehingga arah ketergantungan sudah terbaca sebelum satu pun metode dibuka.

Tabel 2.8 memperlihatkan hasil pembacaan tersebut sebagai contoh.

Isikan hasil pembacaan pada Tabel 2.9.

Jawab tiga pertanyaan berikut dengan menunjuk isi tabel beserta kodenya. Pertama, sebutkan bagian kode mana yang melakukan *violation*, yaitu yang tidak melewati *business layer* melainkan langsung menuju sumber datanya. Sebutkan nama berkas, nama kelas, nama metode, dan baris rujukan yang memungkinkan-nya. Kedua, kedua *violation* tersebut tidak sama beratnya, sehingga jelaskan mana

Tabel 2.7: Petunjuk Latihan 2, metode yang perlu dibuka beserta urutannya

Urutan	Berkas	Yang dibuka dan yang dicari
1	keduanya	rakit_antarmuka, bandingkan jumlah objek yang dirangkai
2	keduanya	AntarmukaKurs.__init__, baca anotasi tipe parameternya
3	open	AntarmukaKurs.tampilkan_kurs_dasar, cari pemanggilan self.repositori
4	open	AntarmukaKurs.tampilkan_kurs_jual, cari pemanggilan yang sama
5	closed	LayananKurs.susun_kurs_jual, catat dua pemeriksaan yang tidak punya pasangan pada versi open
6	keduanya	konstanta LAPISAN, perhatikan layer yang tidak tercantum

Tabel 2.8: Contoh hasil Latihan 2 yang sudah terisi

Berkas	Violation	Rincian
aplikasi_tertutup.py	0	tidak ada
aplikasi_terbuka.py	2	AntarmukaKurs.tampilkan_kurs_dasar dan AntarmukaKurs.tampilkan_kurs_jual memanggil RepositoriKurs langsung

Tabel 2.9: Lembar isian Latihan 2

Berkas	Violation	Rincian
aplikasi_tertutup.py	...	...
aplikasi_terbuka.py	...	...
Setelah metode baru ditambahkan	...	...

yang dapat dibenarkan beserta alasannya. Ketiga, jelaskan apa yang hilang ketika kurs jual diambil langsung dari sumber datanya, lalu sebutkan siapa yang dirugikan.

Latihan 3: Mengukur Biaya Satu Lompatan

Latihan ini mengukur berapa besar waktu yang dihemat dengan melewati pemeriksaan yang dituntut *business layer*. Yang diukur adalah permintaan kurs jual, sebab di situlah kedua jalur benar-benar berbeda. Langkah kerjanya sebagai berikut.

- 1. Buka `sources/chapter02/ukur_lapisan.py`, lalu perhatikan bahwa skrip tersebut menjalankan seratus putaran, bukan satu.
- 2. Jalankan `python ukur_lapisan.py`. Skrip tersebut mencetak baris kema-juan setiap sepuluh putaran, sehingga jalannya terlihat. Catat kedelapan angka pada laporan akhirnya.
- 3. Baca baris *Range* dibagi selisih, yaitu berapa kali *range* di dalam satu jalur lebih besar daripada selisih antar kedua jalur.
- 4. Catat pada berapa putaran jalur *open* tercatat lebih cepat. Bila kedua jalur

sama cepat, angka itu berkisar di sekitar lima puluh.

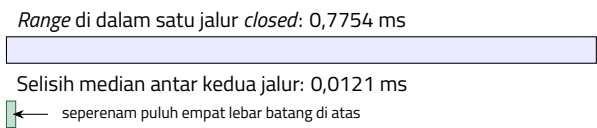
5. Jalankan skrip tersebut sekali lagi, lalu bandingkan kedua hasilnya.

Tabel 2.10 memperlihatkan hasil satu kali pengukuran nyata. Angkanya disalin apa adanya, termasuk kejanggalannya.

Tabel 2.10: Contoh hasil Latihan 3 yang sudah terisi, seratus putaran

Yang dilaporkan	Nilai
Median jalur <i>closed</i> , dengan pemeriksaan	0,7308 ms
Median jalur <i>open</i> , tanpa pemeriksaan	0,7188 ms
Selisih median antar jalur	−0,0121 ms
<i>Range</i> di dalam jalur <i>closed</i>	0,7754 ms
<i>Range</i> di dalam jalur <i>open</i>	0,7451 ms
<i>Range</i> dibagi selisih	64 kali
Putaran jalur <i>open</i> lebih cepat	48 dari 100

Baris keenam itulah kejanggalannya. Selisih antar kedua jalur hanya 0,0121 milidetik, sedangkan dua putaran pada jalur yang sama saja dapat berselisih sampai 0,7754 milidetik. *Range* di dalam satu jalur 64 kali lebih besar daripada selisih yang dicari. Membandingkan satu putaran jalur *closed* dengan satu putaran jalur *open* karena itu hampir tidak berarti. Gambar 2.5 menempatkan kedua angka tersebut pada satu skala.



Gambar 2.5: Selisih antar kedua jalur dan *range* di dalam satu jalur, digambar pada skala yang sama

Pencacahan per putaran memberi kejanggalan kedua. Median keseluruhan jalur *open* memang sedikit lebih kecil, tetapi jalur itu hanya tercatat lebih cepat pada 48 dari 100 putaran, yaitu justru di bawah angka seimbang 50. Kedua ukuran tersebut tidak harus sejalan bila selisihnya jauh lebih kecil daripada *range*. Delapan pengukuran berturut-turut pada mesin yang sama memberi 63, 60, 62, 57, 53, 56, 59, dan 48, sehingga angkanya berayun di sekitar 50. Median mutlaknya sendiri bergeser dari 0,6 sampai 1,9 milidetik mengikuti kesibukan mesin, sehingga yang layak dilaporkan adalah pola perbandingannya, bukan angka mutlaknya.

Asal selisih tersebut tetap jelas. Jalur *open* tidak memakai algoritma yang lebih baik. Jalur itu hanya tidak menjalankan dua pemeriksaan, yaitu memeriksa token pemanggil dan memeriksa peran pemanggil.

Iniilah temuan utama latihan tersebut. Yang dihemat berupa waktu yang kecil dan sulit dipastikan, yaitu sekitar 1,7 persen dari satu permintaan dan tenggelam di

dalam *range* pengukuran. Yang dilepas berupa dua pemeriksaan yang pasti hilang pada seluruh permintaan. Keputusan memakai jalan pintas karena itu tidak dapat diambil dari angka kecepatan, melainkan dari ada tidaknya tuntutan bisnis yang ikut dilewati. Pada kurs dasar jalan pintas dapat dibenarkan, sedangkan pada kurs jual tidak.

Isikan hasil pengukuran pada Tabel 2.11.

Tabel 2.11: Lembar isian Latihan 3

Yang dilaporkan	Nilai
Median jalur <i>closed</i>	...
Median jalur <i>open</i>	...
Selisih median antar jalur	...
<i>Range</i> di dalam jalur <i>closed</i>	...
<i>Range</i> dibagi selisih	...kali
Putaran jalur <i>open</i> lebih cepat	...dari 100
Penghematan dalam persen	...
Tuntutan bisnis yang dilepas	...

Jawab tiga pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, hitung penghematan waktunya dalam persen, lalu bandingkan dengan tuntutan bisnis yang dilepas. Kedua, *range* di dalam satu jalur puluhan kali lebih besar daripada selisih antar kedua jalur, sehingga jelaskan mengapa satu putaran tidak cukup untuk menyimpulkan jalur mana yang lebih cepat, dan apa yang membuat pen-cacahan seratus putaran lebih dapat dipercaya. Ketiga, dengan menggabungkan hasil ketiga latihan, tentukan permintaan mana yang boleh memakai jalan pintas dan mana yang tidak, lalu sebutkan angka yang menjadi dasar keputusannya.

Bab 3

Model-View-*

Tujuan Pembelajaran

1. Menjelaskan pembagian peran pada keempat varian Model-View-*, yaitu MVC, MVP, MVVM, dan MVI, beserta arah ketergantungan di antara komponennya.
2. Membedakan keempat varian tersebut berdasarkan komponen yang dikenal oleh View, lalu menentukan varian mana yang sesuai untuk sebuah kebutuhan.
3. Mengukur jumlah ketergantungan, kemudahan pengujian tanpa antarmuka, dan luas dampak perubahan pada keempat varian.

3.1 Pendahuluan

Antarmuka pengguna berubah jauh lebih sering daripada aturan bisnis. Warna, susunan tombol, dan urutan layar dapat berganti beberapa kali dalam satu semester, sedangkan aturan perhitungannya tetap. Mencampur keduanya di dalam satu berkas membuat setiap perubahan tampilan berisiko merusak perhitungan.

Kelompok pola Model-View-* lahir untuk memisahkan keduanya. Seluruh varian membagi aplikasi menjadi bagian yang menyimpan data, bagian yang menampilkan, dan bagian yang menghubungkan keduanya. Yang membedakan antar varian hanyalah bagaimana bagian penghubung tersebut bekerja, dan komponen mana yang boleh dikenal oleh View.

Bab ini membahas empat varian yang paling banyak dipakai, yaitu Model-View-Controller (MVC), Model-View-Presenter (MVP), Model-View-ViewModel (MVVM), dan Model-View-Intent (MVI). Keempatnya diperbandingkan memakai satu aplika-

si yang sama, sehingga perbedaannya dapat diukur, bukan sekadar diperdebatkan. Fowler [5] menyusun perbandingan sejenis dan memakai istilah *passive view* untuk salah satu bentuk yang dibahas di sini. Pembahasan mengikuti kerangka yang disusun oleh Richards dan Ford [18].

3.2 Peran Ketiga Komponen

Seluruh varian Model-View-* berangkat dari pembagian yang sama. Tabel 3.1 merangkum ketiga peran dasarnya.

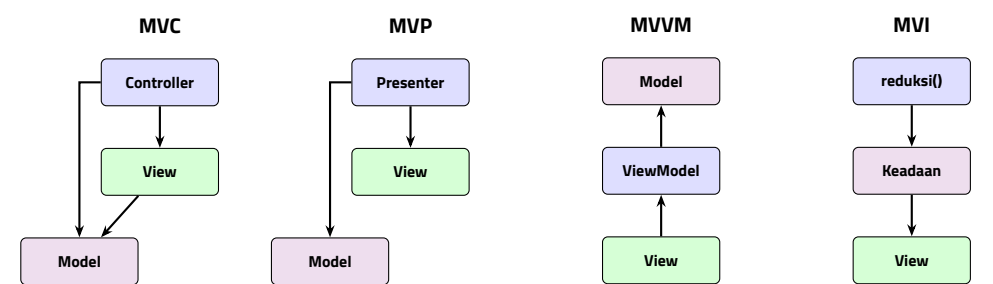
Tabel 3.1: Peran dasar yang dipakai seluruh varian Model-View-*

Peran	Tanggung jawab
Model	Menyimpan data dan menjalankan aturan bisnis
View	Menampilkan data dan menerima tindakan pengguna
Penghubung	Menerjemahkan tindakan pengguna menjadi perubahan pada Model

Nama peran penghubung berbeda pada setiap varian. Pada MVC namanya Controller, pada MVP namanya Presenter, pada MVVM namanya ViewModel, sedangkan pada MVI peran tersebut dipegang oleh sebuah fungsi reduksi, bukan oleh sebuah kelas. Perbedaan nama tersebut tidak penting. Yang menentukan adalah arah ketergantungan di antara ketiganya.

3.3 Empat Varian dan Arah Ketergantungannya

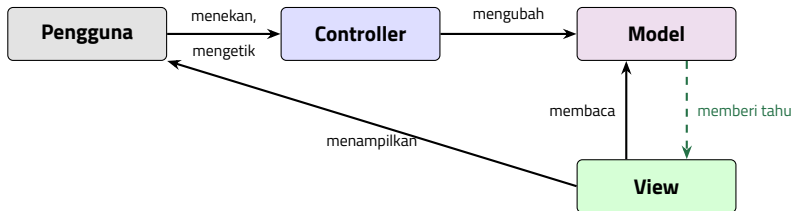
Gambar 3.1 menunjukkan keempat varian berdampingan. Panah menyatakan arah ketergantungan, yaitu komponen yang menunjuk mengenal komponen yang ditunjuk.



Gambar 3.1: Arah ketergantungan pada keempat varian. MVC di sini digambarkan dalam bentuk masa kini, sedangkan bentuk klasiknya berbeda dan dibahas pada Gambar 3.2. Setiap peran diberi warna tersendiri, yaitu ungu untuk Model, biru untuk penghubung, dan hijau untuk View. Letak pengguna dibahas pada gambar setiap varian

3.3.1 Model-View-Controller

MVC punya dua bentuk yang berbeda, dan keduanya sama-sama disebut MVC. Bentuk pertama adalah bentuk asli dari Smalltalk-80 [17, 11], sedangkan bentuk kedua adalah yang dipakai kerangka kerja web dan aplikasi desktop masa kini. Gambar 3.2 menunjukkan bentuk aslinya.



Gambar 3.2: MVC klasik. Controller mengubah Model, lalu Model memberi tahu View, dan View membaca Model untuk menggambar ulang

Pada bentuk klasik, alurnya berputar. Controller menerima tindakan pengguna lalu mengubah Model. Model kemudian memberi tahu seluruh pihak yang mengamatinya, termasuk View. View lalu membaca Model dan menggambar ulang dirinya. Model tidak mengetahui jenis View yang mengamatinya, sebab pemberituannya memakai mekanisme pengamatan.

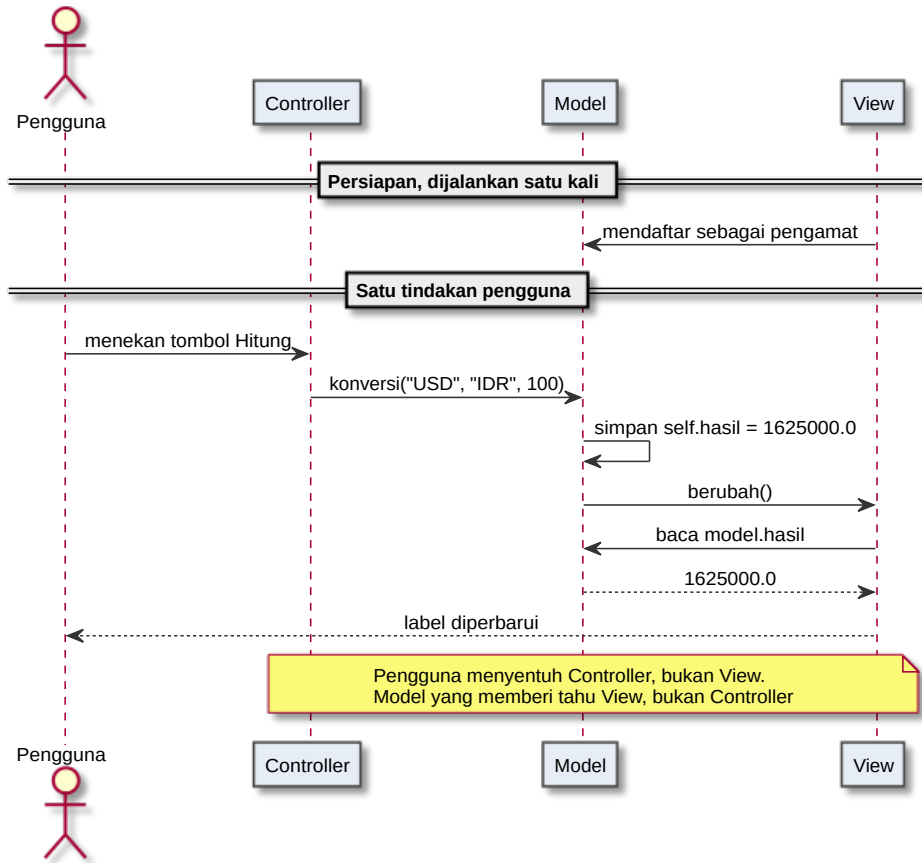
Penelusuran satu tindakan memperjelas urutannya. Pengguna memasukkan 100 USD ke IDR, lalu langkahnya sebagai berikut.

1. View mendaftarkan diri sebagai pengamat Model. Pendaftaran tersebut dijalankan satu kali saat aplikasi disusun, bukan pada setiap tindakan.
2. Pengguna menekan tombol, dan Controller yang menerimanya. Controller menafsirkan sendiri peristiwa tetikus dan papan ketik.
3. Controller memanggil `model.konversi` dengan tiga nilai mentah.
4. Model menghitung, menyimpan hasilnya sebagai `self.hasil`, lalu **memberi tahu seluruh pengamatnya**.
5. View menerima pemberitahuan tersebut, lalu **menarik sendiri** nilai `model.hasil` dan menggambar ulang dirinya.

Gambar 3.3 menunjukkan urutan tersebut sebagai diagram sekuens.

Langkah keempat memuat perbedaan yang paling menentukan. Yang memicu penggambaran adalah Model, bukan Controller. Controller bahkan tidak mengetahui bahwa View sudah menggambar ulang. Susunan seperti itu memungkinkan satu Model diamati banyak View sekaligus, dan seluruhnya diperbarui oleh satu pemberitahuan.

Bentuk kedua muncul karena kerangka kerja web bekerja dengan cara yang

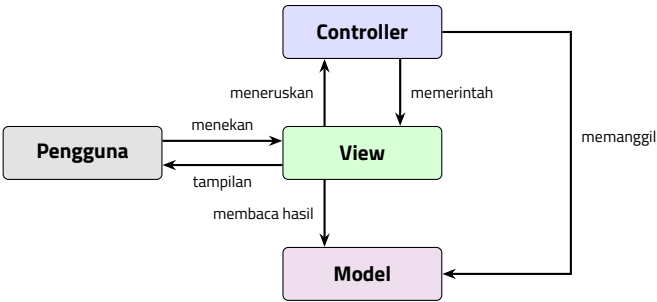


Gambar 3.3: Diagram sekuens MVC klasik. Model yang memicu penggambaran, dan Controller tidak menyentuh View sama sekali

berbeda. Permintaan datang, dilayani, lalu selesai, sehingga tidak ada View yang menetapkan untuk diberi tahu. Gambar 3.4 menunjukkan bentuk tersebut.

Letak pengguna pada bentuk kedua berbeda dari bentuk klasiknya, dan perbedaan itu sering terlewat. Pada bentuk klasik, Controller menafsirkan sendiri peristiwa tetikus dan papan ketik, sehingga pengguna berhadapan langsung dengannya. Pada bentuk masa kini, Controller tidak punya permukaan yang dapat disentuh sama sekali.

Yang disentuh pengguna adalah View. Pada aplikasi web, pengguna menekan tautan pada halaman yang sudah tergambar, lalu perute mengubahnya menjadi permintaan dan mengantarkannya ke Controller. Pada aplikasi desktop, widget dimiliki View, dan penanganan widget itulah yang memanggil Controller. Keduanya berbeda pada perantaranya, tetapi sama pada intinya, yaitu pengguna menyentuh View sedangkan Controller hanya menerima permintaan yang sudah diteruskan.



Gambar 3.4: MVC pada kerangka kerja masa kini. Pengguna menyentuh View, lalu permintaannya diteruskan ke Controller. Controller memanggil Model, lalu memerintah View. View membaca hasil dari Model yang rujukannya sudah diserahkan Controller, dan Model tidak memberi tahu siapa pun

Tabel 3.2 merangkum perbedaan keduanya.

Tabel 3.2: Perbedaan MVC klasik dan MVC pada kerangka kerja masa kini

Hal	MVC klasik	MVC masa kini
Pemicu penggambaran	Model memberi tahu View	Controller memerintah View
Sifat	Menetap, View hidup terus	Sekali jalan, permintaan lalu selesai
Peran Controller	Menangani masukan saja	Menangani masukan dan mengatur alur
Yang disentuh pengguna	Controller, sebab Controller menafsirkan tetikus dan papan ketik	View, lalu diteruskan ke Controller
Pengetahuan Model	Tahu ada yang mengamati	Tidak tahu apa pun

Contoh pada bab ini memakai bentuk masa kini, sebab bentuk itulah yang paling sering ditemui. Ciri khasnya terletak pada konstruktor View, yang menerima Model sebagai argumen. Karena itu View tidak dapat dibuat tanpa Model, dan pengujian logika tampilan tetap menuntut kedua objek tersebut ada. Tombolnya pun dimiliki View, sehingga pengguna menyentuh View lalu View meneruskannya ke Controller.

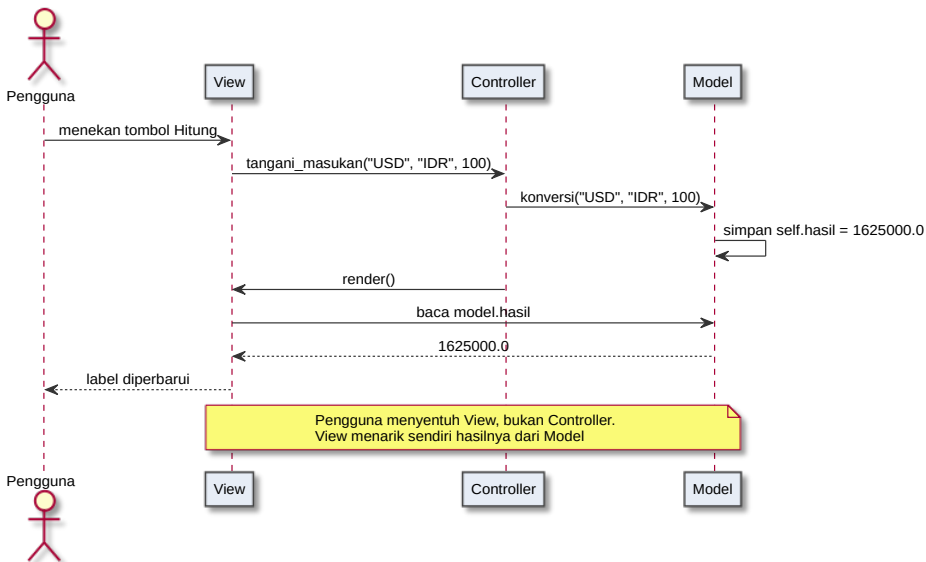
```
1 class View:
2
3     def __init__(self, induk, model):
4         self.model = model
5         self.controller = None
6         self.tombol = tk.Button(induk, text="Hitung",
7                                 command=self.saat_ditekan)
8
9     def saat_ditekan(self):
10         # Widget dimiliki View, sehingga pengguna menyentuh View.
11         self.controller.tangani_masukan("USD", "IDR", 100)
```

Listing 3.1: View pada MVC mengenal Model, sekaligus meneruskan tindakan pengguna, tersedia di `sources/chapter03/mvc.py`

Penelusuran satu permintaan memperjelas apa yang menyeberang antar komponen. Pengguna memasukkan 100 USD ke IDR, lalu urutannya sebagai berikut.

1. Pengguna menekan tombol milik View, lalu View memanggil `controller.tangani_masukan` dengan tiga nilai mentah, yaitu "USD", "IDR", dan 100.
2. Controller memanggil `model.konversi` dengan ketiga nilai tersebut.
3. Model menghitung, lalu **menyimpan hasilnya di dalam dirinya sendiri** sebagai `self.hasil`.
4. Controller memanggil `view.render` tanpa membawa data apa pun.
5. View **menarik sendiri** nilai `model.hasil`, lalu menyusun teks Hasil: 1,625,000.00.

Gambar 3.5 menunjukkan urutan tersebut sebagai diagram sekuens.



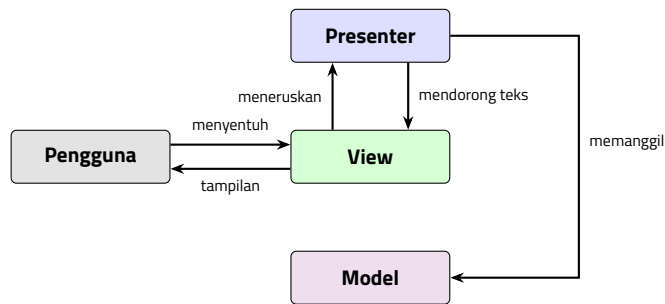
Gambar 3.5: Diagram sekuens MVC. Hasil disimpan di Model, lalu ditarik sendiri oleh View

Ciri khas MVC terletak pada langkah kelima. Hasil tidak dikirimkan kepada View, melainkan diambil View dari Model. Karena itu View wajib mengenal Model.

Perlu satu penegasan agar tidak salah paham. View memang membaca Model, tetapi View tidak mencari sendiri Model yang mana. Rujukannya diserahkan lebih dahulu, yaitu lewat konstruktor pada aplikasi desktop atau lewat objek yang dititipkan Controller pada aplikasi web. Jadi Controller tetap memegang kendali alurnya, sedangkan View hanya membaca isi objek yang sudah diberikan kepadanya.

3.3.2 Model-View-Presenter

MVP diperkenalkan Potel [15] sebagai penyederhanaan MVC. MVP menjadikan View pasif. Presenter mengerjakan seluruh penyusunan teks, lalu mendorong hasilnya ke View. Gambar 3.6 menunjukkan bahwa tidak ada panah dari View ke Model. Ruang di bawah View kosong, dan kekosongan itulah pembeda MVP dari MVC.



Gambar 3.6: MVP, dengan View pasif yang tidak mengenal Model. Pengguna tetap menyentuh View, lalu View meneruskannya ke Presenter

View pada MVP hanya menyimpan teks yang diterimanya. Karena tidak ada perilaku yang perlu ditiru selain menyimpan, View tersebut dapat diganti tiruan sederhana saat diuji.

Bentuk yang dipakai di sini disebut *passive view*, yaitu View yang sama sekali tidak mengenal komponen lain. Banyak penerapan MVP memakai bentuk lain, yaitu View menyimpan rujukan ke Presenter agar dapat meneruskan tindakan pengguna. Pada bentuk tersebut hubungannya menjadi dua arah, dan angka ketergantungannya bertambah satu.

```

1 class View:
2     """Menyimpan teks yang diberikan Presenter, tanpa logika apa pun."""
3
4     def tampilkan(self, teks):
5         self.teks = teks
  
```

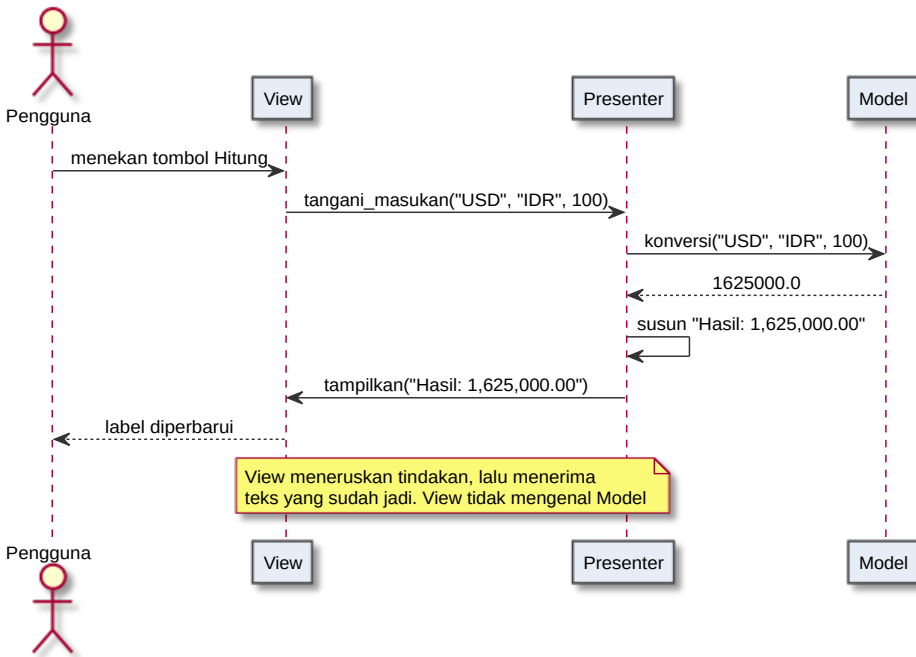
Listing 3.2: View pasif pada MVP, tersedia di `sources/chapter03/mvp.py`

Penelusuran permintaan yang sama pada MVP berbeda mulai langkah ketiga.

1. Presenter menerima tiga nilai mentah.
2. Presenter memanggil `model.konversi`, dan Model **mengembalikan** angka `1625000.0` sebagai nilai balik.
3. Presenter menyusun teks Hasil: `1,625,000.00`.
4. Presenter memanggil `view.tampilkan` dengan **membawa teks yang sudah jadi**.

5. View menyimpan teks tersebut tanpa mengolahnya.

Gambar 3.7 menunjukkan urutan tersebut sebagai diagram sekuens.



Gambar 3.7: Diagram sekuens MVP. Teks yang sudah jadi didorong Presenter ke View

Perbedaannya jelas. Pada MVC hasil disimpan di Model lalu ditarik View, sedangkan pada MVP hasil dikembalikan ke Presenter lalu didorong ke View. View karena itu tidak perlu mengenal Model.

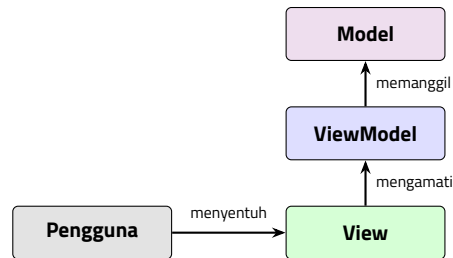
3.3.3 Model-View-ViewModel

MVVM diperkenalkan Gossman [10] bersama kerangka kerja Windows Presentation Foundation. MVVM membalik arah pemberitahuan. ViewModel menyediakan properti yang dapat diamati, lalu View mendaftarkan diri sebagai pengamat. Ketika properti berubah, View memperbarui dirinya sendiri. Gambar 3.8 menunjukkan bahwa ViewModel tidak mengenal View.

Jumlah ketergantungan MVVM sama dengan MVC, yaitu tiga. Arahnya yang berbeda. Pada MVC View mengenal Model, sedangkan pada MVVM View mengenal ViewModel dan ViewModel tidak mengenal View. Karena itu logika tampilan dapat diuji cukup dengan membaca properti, tanpa membuat objek View.

```

1 class View:
2     """Mengikat diri pada properti ViewModel dan menyimpan nilai terbarunya."""
3 
```



Gambar 3.8: MVVM, dengan View yang mengamati properti milik ViewModel

```

4 def __init__(self, view_model):
5     self.terakhir = None
6     view_model.teks.amati(self.saat_berubah)
  
```

Listing 3.3: View mengikat diri pada properti, tersedia di `sources/chapter03/mvvm.py`

Penelusuran permintaan yang sama pada MVVM berbeda pada arah pemberitahuan.

1. ViewModel menerima tiga nilai mentah.
2. ViewModel memanggil `model.konversi` dan menerima `1625000.0`.
3. ViewModel menulis teks ke propertinya lewat `teks.ubah`.
4. Properti tersebut **memanggil seluruh pengamatnya** satu per satu.
5. View menerima teks baru lewat `saat_berubah`, lalu memperbarui dirinya.

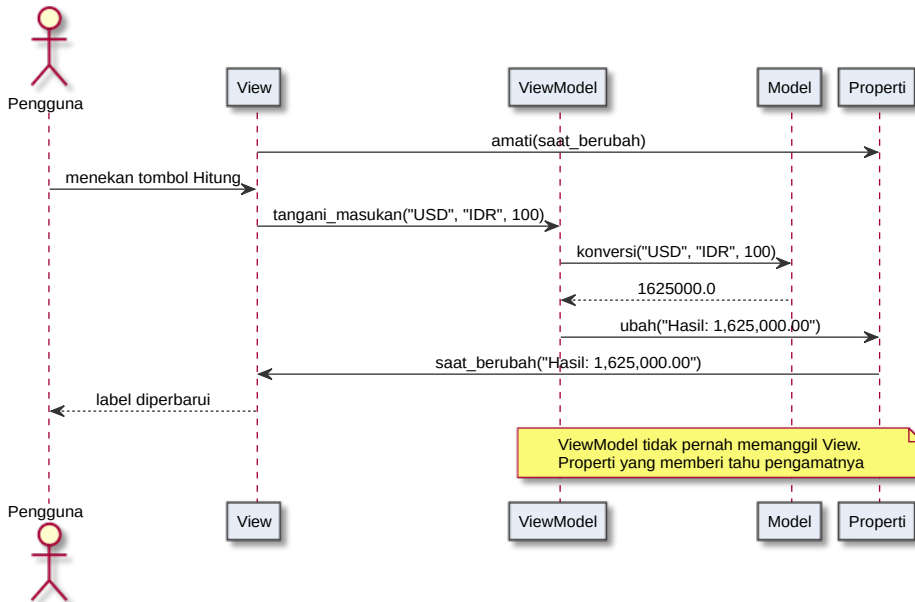
Gambar 3.9 menunjukkan urutan tersebut sebagai diagram sekuens. Perhatikan langkah pertama, yaitu View mendaftarkan diri sebagai pengamat sebelum permintaan datang.

ViewModel tidak pernah memanggil View. ViewModel hanya mengubah nilai properti, dan properti itulah yang memberi tahu siapa pun yang mendaftar. Karena itu satu properti dapat diamati beberapa View sekaligus tanpa ViewModel mengetahuinya.

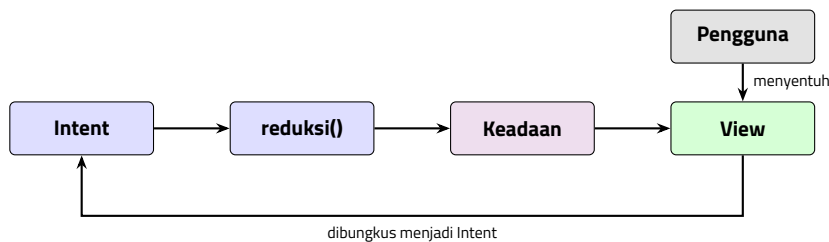
3.3.4 Model-View-Intent

MVI menghapus kelas penghubung sama sekali. Setiap tindakan pengguna dibungkus menjadi Intent, lalu sebuah fungsi reduksi mengubah pasangan keadaan lama dan Intent menjadi keadaan baru yang tidak dapat diubah lagi. Gambar 3.10 menunjukkan susunannya.

Listing 3.4 memperlihatkan fungsi tersebut. Karena fungsi itu murni, seluruh keadaan yang pernah digambar dapat disimpan lalu diputar ulang persis seperti semula. Sifat itu membuat antarmuka yang keluarannya berubah-ubah tetap



Gambar 3.9: Diagram sekuensi MVVM. Properti yang memberi tahu pengamatnya, bukan ViewModel



Gambar 3.10: MVI bersifat searah dan membentuk siklus. View menggambar Keadaan, lalu tindakan pengguna kembali menjadi Intent

dapat diuji secara pasti.

```

1 def reduksi(keadaan, intent):
2     """Mengubah keadaan lama dan sebuah Intent menjadi keadaan baru.
3
4     Fungsi ini murni, artinya keluarannya hanya bergantung pada kedua
5     masukannya.
6     Sifat itulah yang membuat riwayat tampilan dapat diputar ulang.
7     """
8     try:
9         hasil = domain.hitung_konversi(intent.kode_asal, intent.kode_tujuan,
10                                       intent.nominal)
11         return replace(keadaan, teks=f"Hasil: {hasil:,.2f}", sedang_sibuk=False)
12     except (ValueError, LookupError) as kesalahan:
13         return replace(keadaan, teks=f"Galat: {kesalahan}", sedang_sibuk=False)
  
```

Listing 3.4: Fungsi reduksi pada MVI, tersedia di `sources/chapter03/mvi.py`

Agar keempat varian tidak berhenti sebagai nama, Tabel 3.3 menunjukkan isi setiap peran pada aplikasi konversi yang dipakai sepanjang bab ini. Perhatikan bahwa isi Model dan View hampir sama pada keempatnya, sedangkan peran penghubungnya berbeda jauh.

Tabel 3.3: Contoh isi setiap peran pada aplikasi konversi

Varian	Isi penghubungnya	Yang dilakukan View
MVC	Controller meneruskan masukan ke Model	Membaca Model lalu menyusun teks
MVP	Presenter menyusun teks lalu menderongnya	Menyimpan teks yang diterima
MVVM	ViewModel menulis hasil ke properti	Memperbarui diri saat properti berubah
MVI	Fungsi reduksi menghasilkan keadaan baru	Menggambar keadaan yang diterima

Kasus berikut memperlihatkan akibatnya pada pekerjaan sehari-hari. Sebuah tim ingin menguji aturan penolakan nominal nol. Pada MVC, pengujian tersebut tetap menuntut objek View dibuat lebih dahulu, sebab Controller merujuk View. View menuntut Model, dan Model menuntut sambungan data. Pengujian yang seharusnya memeriksa satu aturan akhirnya menyalakan hampir seluruh aplikasi. Pada MVI, pengujian yang sama cukup memanggil satu fungsi reduksi tanpa membuat objek apa pun.

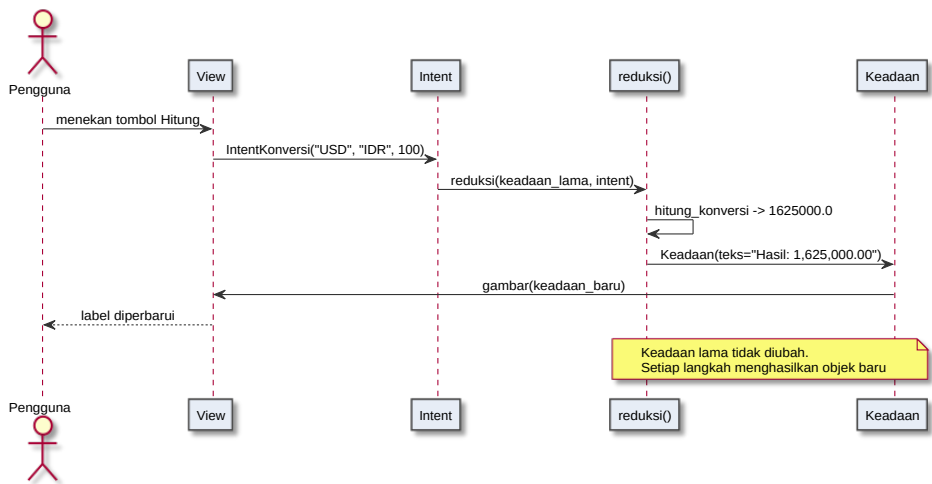
Yang menentukan bukan jumlah kelasnya, melainkan komponen apa yang dituntut View. Itulah sebabnya perbandingan pada bab ini memakai ketergantungan sebagai ukuran, bukan panjang kode.

Penelusuran permintaan yang sama pada MVI tidak memakai pemanggilan metode antar objek sama sekali.

1. Tindakan pengguna dibungkus menjadi `IntentKonversi("USD", "IDR", 100)`.
 2. Fungsi `reduksi` dipanggil dengan dua masukan, yaitu keadaan lama dan `Intent` tersebut.
 3. Di dalamnya `domain.hitung_konversi` menghasilkan `1625000.0`.
 4. Fungsi tersebut **mengembalikan objek Keadaan baru** berisi teks hasil, tanpa mengubah keadaan lama.
 5. View menggambar Keadaan tersebut, lalu menyimpannya ke dalam riwayat.
- Gambar 3.11 menunjukkan urutan tersebut sebagai diagram sekuens.

Seluruh isi tampilan terbungkus dalam satu objek Keadaan. Karena keadaan lama tidak pernah diubah, seluruh riwayat tersimpan utuh dan dapat diputar ulang.

Tabel 3.4 merangkum apa yang menyeberang menuju View pada keempat



Gambar 3.11: Diagram sekuens MVI. Setiap langkah menghasilkan objek baru, tanpa mengubah yang lama

varian.

Tabel 3.4: Apa yang menyeberang menuju View pada keempat varian

Varian	Yang diterima View	Cara View memperolehnya
MVC	Tidak ada, hanya perintah menggambar	Menarik sendiri dari Model
MVP	Teks yang sudah jadi	Didorong Presenter lewat pemanggilan metode
MVVM	Nilai properti yang baru	Diberi tahu properti yang diamatinya
MVI	Satu objek Keadaan yang utuh	Diserahkan sebagai argumen saat digambar

3.4 Produk yang Memakai Setiap Varian

Keempat varian tersebut bukan bahan kuliah semata. Setiap varian punya kerangka kerja atau produk yang memakainya. Tabel 3.5 merangkum contoh yang paling sering ditemui.

Empat hal perlu penjelasan tambahan, sebab keempatnya sering menimbulkan salah paham.

Django tidak memakai penamaan MVC. Django menyebut polanya *Model-Template-View* atau MTV. Berkas yang disebut `views.py` pada Django sebenarnya berperan sebagai Controller, sebab berkas tersebut memilih data mana yang disajikan. Templat Django itulah yang berperan sebagai View. Dokumentasi resminya menyatakan Controller pada Django adalah kerangka kerjanya sendiri, yaitu bagian

Tabel 3.5: Contoh produk dan kerangka kerja untuk setiap varian

Varian	Contoh produk	Catatan
MVC klasik	Smalltalk-80	Bentuk aslinya, dengan Model yang memberi tahu View
MVC masa kini	Ruby on Rails, Laravel, Spring MVC, ASP.NET Core MVC	Controller menerima permintaan dari perute, lalu memerintah View
MVP	Google Web Toolkit, Windows Forms, aplikasi Android sebelum era Jetpack	Presenter mendorong teks jadi ke View yang pasif
MVVM	Windows Presentation Foundation, .NET MAUI, Vue.js versi awal	View mengamati properti pada ViewModel
MVI	Redux, arsitektur Elm, Flutter Bloc	Keadaan diganti utuh oleh fungsi reduksi. Android menganjurkan aliran searah dengan nama UDF, tanpa memakai istilah MVI

yang mengantar permintaan ke fungsi yang tepat [3].

Android tidak menyebut dirinya MVVM. Banyak tulisan menyebut Android memakai MVVM karena ada kelas bernama `ViewModel`. Dokumentasi resminya tidak memakai istilah tersebut sama sekali. Dokumentasi tersebut menyebut `ViewModel` sebagai *state holder*, yaitu pemegang keadaan layar, di dalam susunan berlapis yang terdiri atas lapisan antarmuka, domain, dan data [9]. Kemiripannya dengan MVVM memang ada, sebab View mengamati keadaan dan `ViewModel` tidak mengenal View. Meskipun begitu, penyebutannya sebaiknya tetap hati-hati.

Intent pada Android dan Intent pada MVI memakai gagasan yang sama. Kesamaan namanya bukan kebetulan. Keduanya membungkus niat menjadi objek data, lalu menyerahkan objek tersebut kepada pihak yang menyalurkannya.

Intent untuk mengirim pesan singkat memperjelas hal tersebut. Objek tersebut tidak mengirim apa pun sendiri. Objek tersebut hanya menyatakan adanya niat mengirim pesan ke nomor tertentu. Sistem operasi lalu memilih aplikasi mana yang mengerjakannya, dan pengguna bahkan dapat ikut memilih ketika aplikasinya lebih dari satu. Dokumentasi Android menyebutnya objek pesan untuk meminta tindakan dari komponen aplikasi lain [7]. Intent pada MVI bekerja dengan cara yang sama, hanya saja penyalurnya berupa fungsi reduksi.

Tujuan keduanya pun sama. Sebuah Intent dikirim agar ada keadaan yang berubah, atau agar ada proses tertentu yang dijalankan. Objek itu sendiri tidak mengerjakan apa pun.

Gagasan tersebut dikenal luas dengan nama *command*. Sebuah *command* adalah permintaan agar sistem melakukan tindakan yang mengubah keadaannya. Wujudnya berupa kelas yang hanya berisi data yang diperlukan, semacam *Data Transfer Object* khusus [12]. Rumusan tersebut berlaku untuk keduanya tanpa perubahan sedikit pun. Akibat yang menyertainya juga sama. Pengirim tidak perlu

mengenai pihak yang mengerjakan, permintaannya dapat dicatat dan diperiksa, dan pengerjanya dapat diganti tanpa menyentuh pengirimnya.

Yang berbeda hanyalah tingkat penerapannya, bukan gagasan maupun tujuannya. Dua baris pertama pada Tabel 3.6 karena itu berisi hal yang persis sama untuk kedua kolomnya.

Tabel 3.6: Intent pada Android dan Intent pada MVI, satu gagasan pada dua tingkat penerapan

Hal	Intent pada Android	Intent pada MVI
Gagasannya	Niat dibungkus menjadi objek data	Niat dibungkus menjadi objek data
Tujuannya	Ada keadaan yang berubah atau proses yang dijalankan	Ada keadaan yang berubah atau proses yang dijalankan
Contohnya	Niat mengirim pesan singkat ke satu nomor	Niat mengonversi 100 USD menjadi IDR
Penyalurnya	Sistem operasi Android	Fungsi reduksi
Cakupannya	Melintasi komponen dan aplikasi	Berada di dalam satu layar
Wujud perubahannya	Komponen lain dijalankan, misalnya aplikasi pesan terbuka	Objek keadaan baru dikembalikan

Kesimpulannya perlu ditarik hati-hati. Pemakaian kelas `Intent` tidak dengan sendirinya menjadikan sebuah aplikasi Android memakai MVI. MVI menuntut lebih dari sekadar membungkus niat, yaitu keadaan layar yang diganti oleh fungsi reduksi. Bagian Android yang benar-benar sejajar dengan MVI justru terletak di tempat lain. Dokumentasi Android menganjurkan *unidirectional data flow* atau UDF, yaitu aliran data satu arah. Keadaan mengalir turun menuju antarmuka, sedangkan peristiwa mengalir naik menuju `ViewModel` [8]. Bentuk aliran tersebut sama dengan MVI, meskipun istilah MVI tidak dipakai dalam dokumentasinya.

Redux mewarisi arsitektur Elm. Dokumentasi Redux menyebut Elm sebagai sumber gagasannya. Elm memakai susunan *model view update*, dan fungsi pembaruannya bertanda tangan (`action, state`) => `state`. Fungsi tersebut sama perannya dengan fungsi reduksi pada Redux [16]. Karena itu Redux menjadi contoh MVI yang paling mudah ditemui.

Vue.js pada dokumentasi versi awalnya menyebut setiap instansinya sebagai `ViewModel`, dan karena itu variabelnya dinamai `vm` [20]. Dokumentasi versi terbarunya tidak lagi memakai istilah tersebut. Penyebutan seperti ini memang berubah seiring waktu, sehingga rujukan versi perlu ikut dicatat.

3.5 Ketergantungan View sebagai Pembeda

Satu pertanyaan cukup untuk membedakan keempat varian, yaitu komponen apa yang dikenal oleh View. Jawabannya menentukan seberapa mudah logika tampilan diuji tanpa menjalankan antarmuka sungguhan.

Analogi yang mendekati adalah loket informasi di kampus. Petugas loket yang harus membuka lemari arsip sendiri sulit digantikan, sebab penggantinya harus mengenal isi lemari tersebut. Petugas yang hanya membacakan lembar yang sudah disiapkan bagian lain mudah digantikan siapa pun. View pada MVP dan MVI berperan seperti petugas kedua.

Tabel 3.7 merangkum hasil pemeriksaan nyata memakai `periksa_mv.py`. Kolom terakhir menunjukkan cara pengujian yang paling sedikit menyentuh antar-muka.

Tabel 3.7: Ketergantungan dan cara pengujian keempat varian

Varian	Total	View butuh	Cara pengujian
MVC	3	Model	Membutuhkan View asli
MVP	2	tidak ada	Cukup View tiruan
MVVM	3	ViewModel	Tanpa View, cukup membaca propertinya
MVI	0	tidak ada	Tanpa View, cukup memanggil fungsi reduksi

Angka pada tabel tersebut punya dua bacaan. Pertama, MVI mencatat nol ketergantungan karena penghubungnya berupa fungsi, bukan kelas yang menyimpan rujukan ke komponen lain. Kedua, MVVM mencatat total yang sama dengan MVC, tetapi ketergantungannya menunjuk ke arah yang berbeda. Pada MVC, View mengenal Model. Pada MVVM, View mengenal ViewModel, sedangkan ViewModel tidak mengenal View. Jumlah yang sama karena itu tidak berarti sifatnya sama.

3.6 Ketika Satu Komponen Menggemuk

Pemisahan peran hanya bertahan bila setiap komponen menolak pekerjaan yang bukan miliknya. Bila penolakan itu tidak terjadi, satu komponen menampung hampir semua kode. Komponen lain tinggal menjadi hiasan. Keadaan tersebut punya nama sendiri, dan namanya diambil dari komponen yang menggemuk.

3.6.1 Fat Controller

Fat controller adalah Controller yang menyimpan aturan bisnis. Nasihat resminya sederhana. Controller bertugas menerima permintaan, memanggil pihak yang mengerjakan, lalu menyiapkan balasan [19]. Aturan bisnisnya bukan miliknya.

Listing 3.5 memperlihatkan bentuk yang keliru. Validasi, perhitungan, dan penyusunan teks menumpuk di satu tempat.

```
1 class Controller:
2
3     def tangani_masukan(self, asal, tujuan, nominal):
4         if nominal <= 0:                                # aturan bisnis
5             return "Galat: nominal harus positif"
```

```

6     if nominal > 1_000_000_000:           # aturan bisnis
7         return "Galat: nominal melewati batas"
8     kurs = self.model.cari_kurs(asal, tujuan)
9     hasil = nominal * kurs                # perhitungan bisnis
10    return f"Hasil: {hasil:,.2f}"         # penyusunan tampilan

```

Listing 3.5: Controller yang menggemuk, sebab aturan bisnisnya ikut ditulis di dalamnya

Akibatnya ada tiga. Pertama, aturan tersebut tidak dapat dipakai ulang. Pintu masuk lain seperti antarmuka baris perintah atau penjadwal harus menyalin ulang aturan yang sama. Kedua, pengujiannya menuntut konteks permintaan, padahal yang diuji hanyalah perkalian dan perbandingan angka. Ketiga, aturan yang sama tersebar di banyak Controller, sehingga satu perubahan kebijakan menuntut penyuntingan di banyak tempat.

Perbaikannya adalah memindahkan kedua pemeriksaan dan perkaliannya ke Model, lalu menyisakan pemanggilan saja di Controller. Bab 2 sudah memakai pola tersebut pada `hitung_konversi`.

Pada pemrograman aplikasi iOS, gejala yang sama dikenal dengan plesetan *Massive View Controller*. Singkatan MVC dibaca ulang karena View dan Controller menyatu, lalu keduanya menampung seluruh kode.

3.6.2 Fat Model

Fat model adalah kebalikannya. Nasihat *skinny controller*, *fat model* memang benar sebagai arah, tetapi sering dibaca berlebihan. Model lalu menampung empat hal sekaligus, yaitu aturan bisnis, akses basis data, penyusunan teks tampilan, dan pengiriman pemberitahuan.

Model seperti itu punya banyak alasan untuk berubah. Perubahan tata letak kolom basis data menyentuhnya. Perubahan format tanggal menyentuhnya. Perubahan kebijakan diskon juga menyentuhnya. Kelas dengan ribuan baris biasanya lahir dari sini.

Perbaikannya bukan mengembalikan kode ke Controller. Perbaikannya adalah membagi Model menjadi beberapa bagian, misalnya bagian aturan domain, bagian akses penyimpanan, dan bagian penyusunan teks. Bab 4 membahas pemisahan tersebut lebih jauh memakai port dan adapter.

3.6.3 Fat View

Fat view adalah View yang menyimpan logika. Gejalanya berupa percabangan dan perhitungan di dalam templat. Contohnya penghitungan diskon, penentuan peringkat, atau pemilihan pesan berdasarkan banyak syarat.

Kerugiannya jelas. Logika di dalam templat sulit diuji tanpa menggambar antarmuka. Logika tersebut juga mudah tersalin ke templat lain, sebab menyalin lebih cepat daripada memindahkannya ke tempat yang benar. MVP dan MVI menekan gejala ini secara bawaan, sebab View pada kedua varian tersebut hanya menerima hasil yang sudah jadi.

Tabel 3.8 merangkum ketiga gejala tersebut.

Tabel 3.8: Gejala komponen yang menggemuk beserta perbaikannya

Gejala	Tanda yang terlihat	Perbaikan
Fat controller	Validasi dan perhitungan ada di Controller	Pindahkan aturannya ke Model
Fat model	Satu kelas mengurus aturan, basis data, dan format	Bagi menjadi beberapa bagian menurut alasan berubahnya
Fat view	Percabangan dan perhitungan ada di templat	Susun teksnya di Presenter atau di fungsi reduksi

Satu catatan penting menutup bagian ini. Alat `periksa_mv.py` pada bab ini mengukur arah ketergantungan, bukan ukuran komponennya. Sebuah program bisa saja mencatat angka yang rapi, tetapi seluruh aturannya menumpuk di satu kelas. Arah panah yang benar karena itu belum menjamin pembagian kerja yang sehat. Jumlah baris dan jumlah method perlu ikut dilihat.

3.7 Kelebihan dan Kekurangan

Tabel 3.9 merangkum untung rugi keempat varian tersebut.

Tabel 3.9: Perbandingan keempat varian Model-View-*

Varian	Kelebihan	Kekurangan
MVC	Paling sederhana, banyak didukung framework	View mengenal Model, sehingga sulit diuji terpisah
MVP	View pasif dan mudah diganti tiruan	Presenter mudah membengkak karena memuat seluruh penyusunan teks
MVVM	Pembaruan tampilan berjalan otomatis lewat pengamatan	Membutuhkan mekanisme properti yang dapat diamati
MVI	Keadaan tetap, riwayat dapat diputar ulang	Setiap tindakan harus dibungkus menjadi Intent, sehingga kode bertambah

Pemilihan varian jarang bebas sepenuhnya. Framework antarmuka biasanya sudah menganjurkan satu varian tertentu, dan melawan anjuran tersebut berbiaya mahal. Yang tetap dapat dipilih adalah seberapa banyak logika yang dititipkan ke View. Semakin sedikit logika di dalam View, semakin besar bagian aplikasi yang dapat diuji tanpa menjalankan antarmuka.

3.8 Ringkasan

Kelompok pola Model-View-* memisahkan tampilan dari aturan bisnis. Seluruh varian membagi aplikasi menjadi bagian yang menyimpan data, bagian yang menampilkan, dan bagian yang menghubungkan keduanya. Perbedaan antar varian terletak pada cara bagian penghubung bekerja, bukan pada tujuannya.

Pembeda yang paling menentukan adalah komponen yang dikenal oleh View. Pada MVC View mengenal Model, pada MVVM View mengenal ViewModel, sedangkan pada MVP dan MVI View tidak mengenal apa pun. Semakin sedikit yang dikenal View, semakin besar bagian aplikasi yang dapat diuji tanpa menjalankan antarmuka.

Pengukuran pada bab ini menunjukkan jumlah ketergantungan saja tidak cukup untuk menilai sebuah varian. MVC dan MVVM sama-sama mencatat tiga ketergantungan, tetapi arahnya berbeda, dan perbedaan arah itulah yang menentukan kemudahan pengujiannya. Angka perlu dibaca bersama arahnya.

MVI mencatat nol ketergantungan karena penghubungnya berupa fungsi murni. Sifat murni tersebut memberi keuntungan tambahan, yaitu seluruh keadaan yang pernah digambar dapat disimpan lalu diputar ulang. Harganya, setiap tindakan pengguna harus dibungkus menjadi Intent, sehingga jumlah kode bertambah.

Pemilihan varian pada praktiknya lebih sering ditentukan oleh framework yang dipakai daripada oleh pertimbangan bebas. Yang tetap dapat dikendalikan adalah banyaknya logika yang dititipkan ke View. Keputusan itu diambil dengan melihat angka ketergantungan beserta arahnya, bukan dengan mengikuti kebiasaan.

Nama pola perlu dibaca dengan hati-hati. Django menyebut polanya MTV, Android menyebut kelas penghubungnya *state holder*, dan Redux mewarisi arsitektur Elm. Kelas *Intent* pada Android justru memakai gagasan yang sama dengan *Intent* pada MVI, yaitu niat yang dibungkus menjadi objek data. Kesamaan gagasan tersebut tetap tidak berarti Android memakai MVI, sebab tingkat penerapannya berbeda. Sebutan yang beredar luas karena itu perlu diperiksa terhadap dokumentasi resminya.

Arah ketergantungan yang benar juga belum cukup. Satu komponen tetap dapat menampung hampir semua kode, dan gejalanya dikenal sebagai *fat controller*, *fat model*, atau *fat view*. Ukuran komponen karena itu perlu dilihat bersama arahnya.

3.9 Latihan

Ketiga latihan berikut memakai satu basis kode yang sama di `sources/chapter03/`, tetapi menyasar tiga masalah yang berbeda. Latihan 1 memeriksa

siapa mengenal siapa. Latihan 2 memeriksa kemudahan pengujian tanpa antarmuka. Latihan 3 mengukur luas dampak satu permintaan perubahan. Penilaian didasarkan pada kelengkapan tabel hasil dan ketepatan jawaban analisisnya. Hasil setiap mahasiswa dapat berbeda, dan yang dilaporkan adalah pola perbandingannya.

Latihan bab ini menampilkan jendela, sehingga modul `tkinter` harus tersedia. Skrip pada Listing 3.6 menyiapkan `venv` seperti pada bab sebelumnya, lalu memeriksa modul tersebut dan memberi peringatan bila belum ada. Pada Debian dan Ubuntu, modulnya berada di paket `python3-tk` yang dipasang terpisah. Ketiga perintah dijalankan dari akar repositori, sedangkan perpindahan ke direktori bab dilakukan sesudahnya.

Listing 3.6: Menyiapkan `venv` dan memastikan `tkinter` tersedia, skrip tersedia di `sources/siapkan_venv.sh`

```
1 bash sources/siapkan_venv.sh
2 source .venv/bin/activate
3 python -c "import tkinter"
```

Baris terakhir tidak mencetak apa pun bila modulnya tersedia. Bila modulnya belum ada, baris tersebut berhenti dengan `ModuleNotFoundError`, dan paket sistemnya perlu dipasang lebih dahulu.

Latihan 1: Memetakan Ketergantungan Antar Komponen

Latihan ini membuktikan bahwa keempat varian berbeda pada arah ketergantungannya, bukan hanya pada nama komponennya. Langkah kerjanya sebagai berikut.

1. Jalankan skrip pemeriksaan seperti pada Listing 3.7.
2. Catat baris `Total ketergantungan` dan `Ketergantungan View` untuk keempat varian.
3. Buka keempat berkas varian, lalu tunjukkan baris konstruktor yang menjadi sumber setiap ketergantungan tersebut.
4. Gambarkan arah panah antar komponen untuk keempat varian.

Listing 3.7: Menjalankan pemeriksaan ketergantungan

```
1 cd sources/chapter03
2 python periksa_mv.py
```

Tabel 3.10 memperlihatkan hasil satu kali pemeriksaan nyata.

Isikan hasil pemeriksaan pada Tabel 3.11.

Jawab dua pertanyaan berikut dengan menunjuk angka pada tabel. Pertama,

Tabel 3.10: Contoh hasil Latihan 1 yang sudah terisi

Varian	Total	View butuh	Komponen lain
MVC	3	1	Controller butuh Model dan View
MVI	0	0	Penghubungnya berupa fungsi

Tabel 3.11: Lembar isian Latihan 1

Varian	Total	View butuh	Komponen lain
MVC	...	...	...
MVP	...	...	...
MVVM	...	...	...
MVI	...	...	...

MVC dan MVVM mencatat total yang sama, sehingga jelaskan mengapa keduanya tetap berbeda sifatnya. Kedua, jelaskan penyebab MVI mencatat nol ketergantungan.

Latihan 2: Menguji Tanpa Antarmuka

Latihan ini membuktikan hubungan antara ketergantungan View dan kemudahan pengujian. Langkah kerjanya sebagai berikut.

- 1. Jalankan `python uji_tanpa_antarmuka.py`, lalu catat cara uji yang dipakai setiap varian.
- 2. Buka `sources/chapter03/uji_tanpa_antarmuka.py`, lalu bandingkan isi keempat fungsi ujinya.
- 3. Coba tulis ulang pengujian MVC tanpa membuat objek View, lalu catat apa yang menghalangi.
- 4. Hitung berapa baris kode tambahan yang dibutuhkan setiap varian untuk dapat diuji.

Tabel 3.12 memperlihatkan hasil satu kali pengujian nyata.

Tabel 3.12: Contoh hasil Latihan 2 yang sudah terisi

Varian	Lulus	Cara uji
MVC	ya	Membutuhkan View asli
MVI	ya	Tanpa View, cukup memanggil fungsi reduksi

Isikan hasil pengujian pada Tabel 3.13.

Jawab dua pertanyaan berikut. Pertama, cocokkan kolom cara uji pada latihan ini dengan kolom ketergantungan View pada Latihan 1, lalu jelaskan hubungan antara keduanya. Kedua, sebutkan varian yang paling murah diuji beserta alasannya

Tabel 3.13: Lembar isian Latihan 2

Varian	Lulus	Cara uji dan baris tambahan
MVC	...	...
MVP	...	...
MVVM	...	...
MVI	...	...

menurut angka yang diperoleh.

Latihan 3: Mengukur Luas Dampak Perubahan

Latihan ini membuktikan berapa banyak berkas yang tersentuh oleh satu permintaan perubahan yang sama. Permintaannya adalah menambahkan baris kedua pada tampilan yang menyebutkan nilai kurs yang dipakai. Langkah kerjanya sebagai berikut.

- 1. Catat lebih dahulu berkas apa saja yang menurut perkiraan akan berubah pada setiap varian.
- 2. Terapkan perubahan tersebut pada keempat varian, satu per satu.
- 3. Jalankan kembali `python uji_tanpa_antarmuka.py` untuk memastikan keempatnya masih berjalan.
- 4. Hitung berkas, kelas, dan baris yang berubah pada setiap varian memakai perintah pada Listing 3.8.

Listing 3.8: Menghitung baris yang berubah

```
1 cd sources/chapter03
2 git diff --stat mvc.py mvp.py mvvm.py mvi.py
```

Tabel 3.14 memperlihatkan bentuk pengisian yang diharapkan, memakai angka perkiraan sebagai contoh.

Tabel 3.14: Contoh bentuk pengisian Latihan 3

Varian	Kelas	Baris	Komponen yang berubah
MVC	2	6	Model dan View
MVI	2	5	Keadaan dan fungsi reduksi

Isikan hasil pengukuran pada Tabel 3.15.

Jawab tiga pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, varian mana yang paling sedikit tersentuh, dan apakah hasilnya sesuai perkiraan awal. Kedua, jelaskan mengapa varian dengan ketergantungan paling sedikit belum tentu yang paling sedikit berubah. Ketiga, dengan menggabungkan hasil

Tabel 3.15: Lembar isian Latihan 3

Varian	Kelas	Baris	Komponen yang berubah
MVC	...	...	...
MVP	...	...	...
MVVM	...	...	...
MVI	...	...	...
Perkiraan awal yang meleset	...		

ketiga latihan, tentukan varian yang dipilih untuk aplikasi tersebut beserta angka yang menjadi dasar keputusannya.

Bab 4

Ports and Adapters

Tujuan Pembelajaran

1. Menjelaskan peran port dan adapter serta arah ketergantungan yang membuat logika inti bebas dari teknologi penyimpanan.
2. Membedakan port masukan dari port keluaran, lalu menentukan komponen mana yang menjadi adapter pada sebuah sistem.
3. Mengukur kemandirian logika inti, biaya penggantian adapter, dan cakupan pengujian tanpa infrastruktur.

4.1 Pendahuluan

Bab 2 menunjukkan bahwa *layered architecture* menjaga arah panggilan tetap ke bawah. Aturan tersebut menahan perubahan di satu lapisan, tetapi tidak menghilangkan satu persoalan. Lapisan bisnis tetap memanggil lapisan persistensi, sehingga lapisan bisnis tetap mengenal keberadaan penyimpanan.

Ports and adapters, yang juga disebut *hexagonal architecture*, menyelesaikan persoalan tersebut dengan membalik arah ketergantungan. Logika inti tidak lagi memanggil penyimpanan. Logika inti justru menyatakan apa yang dibutuhkannya dalam bentuk antarmuka, lalu pihak lain yang menyediakannya. Antarmuka itu disebut port, dan pengisinya disebut adapter [2].

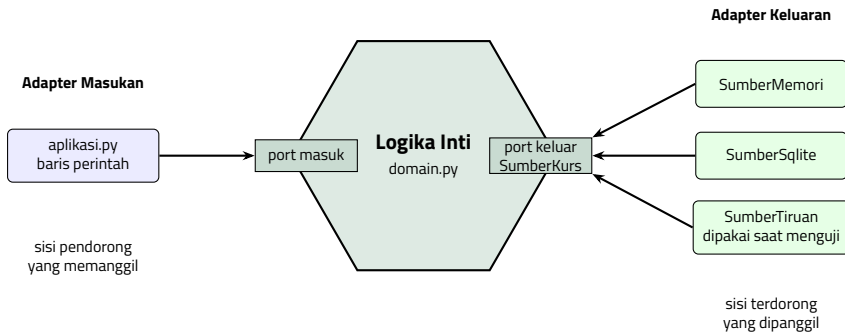
Pembalikan tersebut menghasilkan satu klaim yang tidak biasa, sebab klaim itu menyebut angka yang pasti. Jumlah teknologi penyimpanan yang disebut oleh logika inti harus nol. Klaim yang menyebut angka dapat dibuktikan salah, dan bab ini membuktikannya lewat pengukuran. Pembahasan mengikuti kerangka yang disusun oleh Richards dan Ford [18], serta tulisan asli Cockburn [2] yang

memperkenalkan istilah port dan adapter.

4.2 Port dan Adapter

Susunan ini lebih dikenal dengan nama *hexagonal architecture*, sebab Cockburn menggambarannya sebagai sebuah segi enam [2]. Bentuk segi enamnya sendiri tidak punya arti khusus. Bentuk itu dipilih hanya agar tersedia beberapa sisi, sehingga setiap sisi dapat memuat satu port.

Gambar 4.1 menunjukkan susunan dasarnya. Logika inti berada di tengah, dan seluruh panah ketergantungan menunjuk ke dalam. Karena itu tidak ada panah yang keluar dari logika inti menuju adapter mana pun.



Gambar 4.1: Bentuk heksagonal. Logika inti berada di dalam, port menempel pada sisinya, dan adapter berada di luar. Sisi kiri mendorong, sedangkan sisi kanan didorong

Gambar tersebut memuat tiga bagian yang perlu dibedakan. Bagian pertama adalah isi segi enamnya, yaitu logika inti. Bagian kedua adalah port yang menempel pada sisinya. Bagian ketiga adalah adapter yang berada di luar, dan hanya bagian inilah yang menyebut nama teknologi.

Adapter berada di kedua sisi, bukan hanya di satu sisi. Sisi kiri memuat adapter masukan, dan pada contoh bab ini adapter tersebut adalah `sources/chapter04/aplikasi.py` yang dijalankan dari baris perintah. Sisi kanan memuat adapter keluaran, yaitu ketiga pengisi port `SumberKurs`. Setiap port karena itu selalu berpasangan dengan adapternya sendiri.

Kedua sisinya juga tidak setara, dan perbedaannya penting. Sisi kiri disebut sisi pendorong, sebab pihak di sana memanggil logika inti, misalnya pengguna atau berkas uji. Sisi kanan disebut sisi terdorong, sebab pihak di sana justru dipanggil oleh logika inti, misalnya basis data. Arah panggilannya berlawanan, sedangkan arah ketergantungannya sama-sama menunjuk ke dalam.

Satu adapter pada gambar tersebut belum disebut sebelumnya, yaitu

SumberTiruan. Adapter itu dipasang hanya saat pengujian, dan keberadaannya menjelaskan mengapa susunan ini berguna. Logika inti tidak dapat membedakan ketiga adapter tersebut, sehingga menguji tanpa basis data menjadi mungkin. Latihan 3 pada bab ini membuktikan hal tersebut dengan angka.

Port adalah antarmuka yang ditulis oleh logika inti untuk menyatakan kebutuhannya. Port tidak menyebut teknologi apa pun. Port hanya menyebut kelakuan yang dibutuhkan, misalnya kemampuan mencari nilai kurs.

Adapter adalah pengisi port tersebut untuk satu teknologi tertentu. Satu port dapat diisi oleh banyak adapter sekaligus, misalnya satu untuk penyimpanan dalam memori dan satu lagi untuk basis data.

Port dibedakan menjadi dua arah. **Port masukan** dipakai pihak luar untuk memanggil logika inti, misalnya antarmuka pengguna atau Application Programming Interface (API). **Port keluaran** dipakai logika inti untuk meminta sesuatu dari luar, misalnya membaca data. Bab ini menitikberatkan port keluaran, sebab di situlah ketergantungan pada teknologi paling sering bocor.

Penamaan tersebut kerap disalahpahami. Kata masuk dan keluar menunjuk arah panggilan, bukan arah data. Port masukan berarti panggilannya datang dari luar, sedangkan port keluaran berarti panggilannya berangkat dari logika inti. Data pada keduanya tetap mengalir dua arah, sebab setiap panggilan membawa argumen lalu mengembalikan nilai.

Port keluaran karena itu tidak terbatas pada pembacaan. Sebuah port penyimpanan lazimnya memuat simpan dan cari sekaligus, sehingga port tersebut menulis sekaligus membaca. Letaknya tetap di sisi terdorong, sebab yang memulai panggilan adalah logika inti. Port pada contoh bab ini kebetulan hanya memuat satu metode pembacaan, sebab aplikasinya memang hanya membaca kurs.

Satu port dapat diisi banyak adapter, dan setiap sumber menuntut adapternya sendiri. Kamus di dalam memori, berkas SQLite, dan tiruan untuk pengujian adalah tiga sumber yang berbeda, sehingga tiga adapter dibutuhkan. Hubungan sebaliknya tidak berlaku, sebab satu adapter hanya mengisi satu port. Tabel 4.1 merangkum hubungan tersebut.

Listing 4.1 memperlihatkan port keluaran pada contoh bab ini. Port tersebut hanya menyebut satu metode, dan tidak menyebut basis data sama sekali.

```
1 class SumberKurs(Protocol):
2     """Port keluaran yang dibutuhkan logika inti.
3
4     Siapa pun yang menyediakan metode ini dapat dipasang, baik penyimpanan
5     dalam
6     memori, basis data, maupun tiruan untuk pengujian.
7     """
8     def nilai(self, kode_asal, kode_tujuan):
```

Tabel 4.1: Hubungan antara port dan adapter

Pertanyaan	Jawabannya
Satu port diisi berapa adapter	Boleh banyak. Contoh bab ini memakai tiga, yaitu memori, SQLite, dan tiruan
Satu adapter mengisi berapa port	Satu
Berapa adapter yang aktif saat program berjalan	Satu untuk tiap port, dan pemilihnya berkas perangkat
Apa yang menentukan sisi sebuah port	Pihak yang memulai panggilan, bukan arah datanganya
Apakah port keluaran hanya membaca	Tidak. Port penyimpanan lazimnya menulis sekaligus membaca

9

"""Mengembalikan nilai kurs, atau None bila pasangannya tidak ada."""

Listing 4.1: Port keluaran yang ditulis oleh logika inti, tersedia di `sources/chapter04/domain.py`

Susunan berkasnya terlihat lebih jelas sebagai diagram kelas. Gambar 4.2 menunjukkan seluruh kelas pada contoh bab ini beserta arah ketergantungannya.

Empat hal pada gambar tersebut perlu diperhatikan. Pertama, port berada di dalam kotak hijau, bersama logika intinya. Letak itu bukan kebetulan. Port ditulis oleh pihak yang membutuhkannya, bukan oleh pihak yang mengisinya, dan justru letak itulah yang membalik arah ketergantungannya.

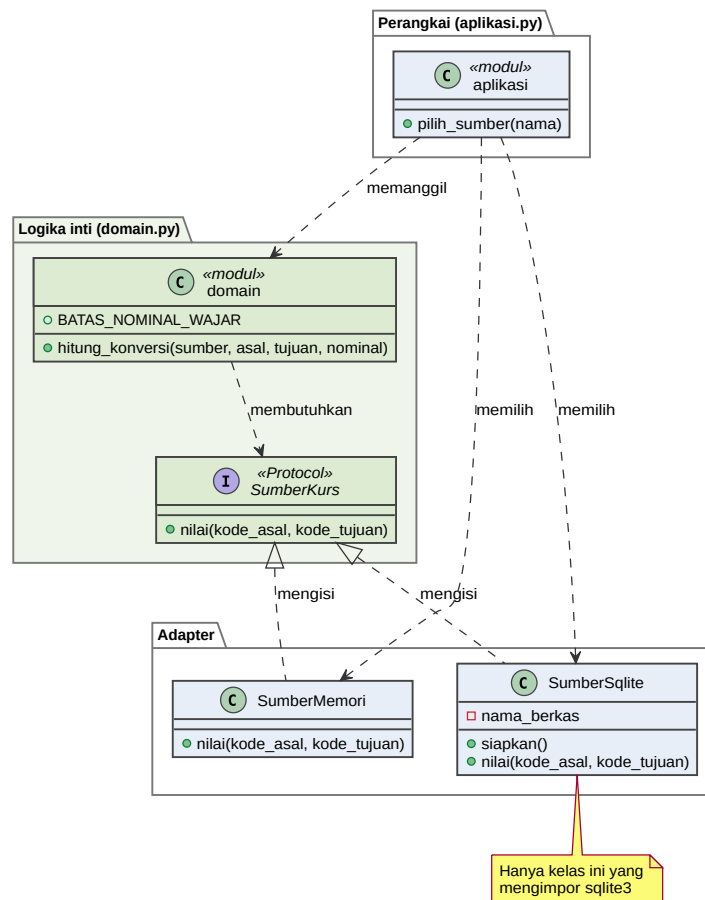
Kedua, kedua adapter menunjuk ke atas menuju port, memakai panah segitiga kosong yang berarti pengisian. Panahnya masuk ke kotak hijau, dan tidak ada satu pun panah yang keluar dari kotak hijau menuju adapter. Sifat itulah yang diperiksa oleh `periksa_port.py`.

Ketiga, hanya aplikasi yang menunjuk ke ketiga kotak sekaligus. Berkas perangkat memang harus mengenal semuanya, sebab berkas itulah yang memasangkan. Ketergantungan terhadap teknologi karena itu terkumpul di satu tempat, dan tidak tersebar.

Keempat, `SumberSqlite` punya satu metode tambahan bernama `siapkan`, sedangkan port tidak menyebut metode tersebut. Adapter memang boleh menyediakan lebih banyak daripada yang diminta port. Yang dilarang adalah sebaliknya, yaitu logika inti memanggil metode yang tidak tercantum pada portnya. Bila panggilan itu terjadi, logika inti terikat pada satu adapter tertentu, dan adapter lain tidak lagi dapat dipasang.

4.3 Menerima Adapter sebagai Argumen

Port saja belum cukup. Logika inti juga tidak boleh membuat adapternya sendiri. Bila logika inti memanggil konstruktor sebuah adapter, logika inti kembali menge-



Gambar 4.2: Diagram kelas port dan adapter. Kotak hijau memuat logika inti beserta portnya, dan tidak ada panah yang keluar dari kotak tersebut menuju adapter

nal teknologinya, dan seluruh manfaatnya hilang.

Analogi yang mendekati adalah stopkontak listrik di dinding. Stopkontak menentukan bentuk colokan yang diterimanya, tetapi tidak menentukan alat apa yang dicolokkan. Kipas angin, lampu, dan pengisi daya telepon sama-sama dapat dipasang. Dinding tidak perlu diubah setiap kali alatnya berganti.

Karena itu adapter diserahkan dari luar sebagai argumen. Listing 4.2 memperlihatkan caranya. Parameter pertama adalah sumber kurs, dan fungsi tersebut tidak pernah tahu adapter mana yang sedang dipakainya.

```

1 def hitung_konversi(sumber, kode_asal, kode_tujuan, nominal):
2     """Menghitung hasil konversi setelah memeriksa dua aturan bisnis.
3
4     Sumber kurs diterima sebagai argumen, bukan dibuat di dalam fungsi. Karena

```

```
itu
5  fungsi ini tidak pernah mengetahui teknologi penyimpanan yang dipakai.
6  """
7  if nominal <= 0:
8      raise ValueError("Nominal harus lebih besar dari nol")
9  if nominal > BATAS_NOMINAL_WAJAR:
10     raise ValueError("Nominal melewati batas wajar")
11     kurs = sumber.nilai(kode_asal, kode_tujuan)
12     if kurs is None:
13         raise LookupError(f"Kurs {kode_asal} ke {kode_tujuan} tidak tersedia")
14     return nominal * kurs
```

Listing 4.2: Adapter diterima sebagai argumen, tersedia di `sources/chapter04/domain.py`

Penentuan adapter dipindahkan ke satu berkas perangkatai. Pada contoh bab ini berkas tersebut adalah `sources/chapter04/aplikasi.py`. Hanya berkas itulah yang mengetahui kedua adapter sekaligus.

Tabel 4.2 menunjukkan isi setiap bagian pada aplikasi yang dipakai sepanjang bab ini. Perhatikan bahwa hanya kolom terakhir yang menyebut nama teknologi.

Tabel 4.2: Contoh isi setiap bagian pada aplikasi kurs

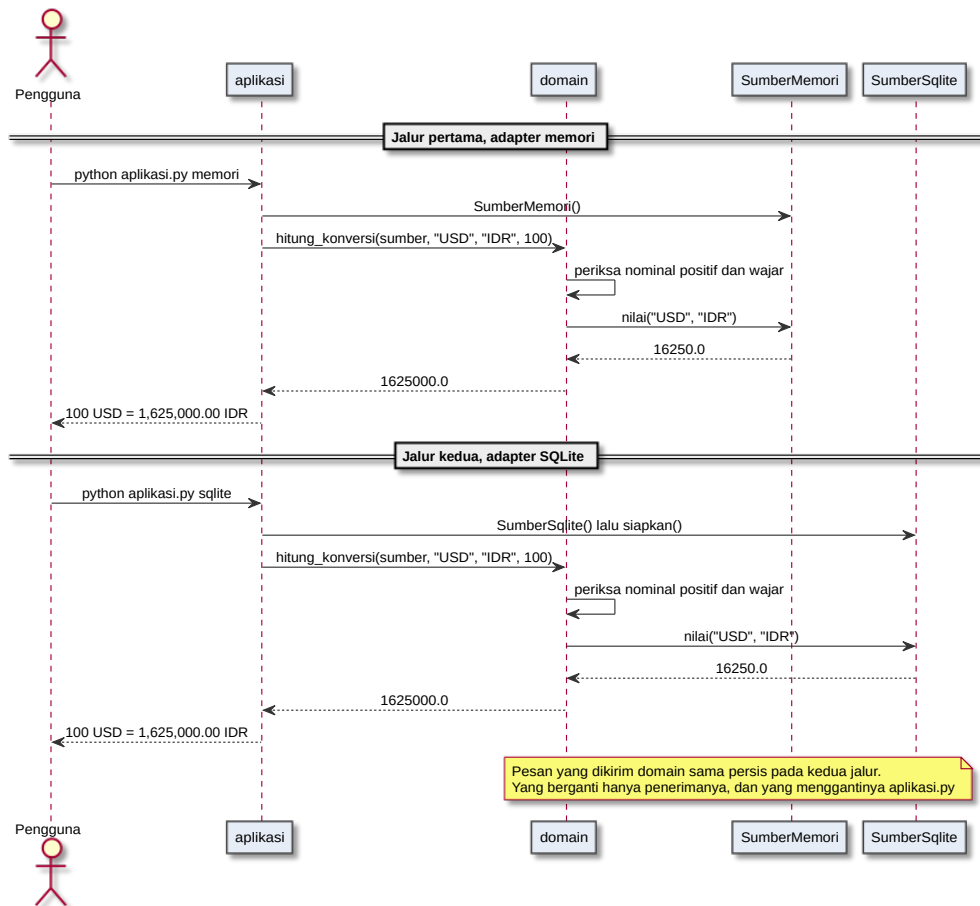
Bagian	Contoh isinya	Menyebut teknologi
Logika inti	Menolak nominal nol, mengalikan nominal dengan kurs	Tidak
Port	Menyatakan kebutuhan berupa satu metode pencari kurs	Tidak
Adapter memori	Menyediakan kurs dari kamus di dalam program	Ya, kamus Python
Adapter SQLite	Menjalankan kueri lalu mengembalikan nilainya	Ya, modul sqlite3
Berkas perangkatai	Memilih adapter mana yang dipasang	Ya, keduanya

Klaim tersebut baru meyakinkan bila dilihat saat program berjalan. Gambar 4.3 menunjukkan dua kali perjalanan aplikasi yang sama, satu memakai adapter memori dan satu memakai adapter SQLite.

Bagian tengah kedua jalur tersebut layak dibandingkan baris demi baris. Logika inti mengirim pesan yang sama persis, yaitu `nilai("USD", "IDR")`. Nilai balikkannya pun sama, yaitu `16250.0`. Perkalian yang dijalankan sesudahnya karena itu tidak berubah sedikit pun.

Yang berganti hanya penerimanya, dan penggantinya adalah `sources/chapter04/aplikasi.py`. Logika inti tidak pernah memilih, sebab adapternya sudah diserahkan sebagai argumen sebelum fungsinya dipanggil.

Satu perbedaan memang terlihat, yaitu pemanggilan siapakan pada jalur kedua. Pemanggilan tersebut dijalankan oleh berkas perangkatai, bukan oleh logika inti.



Gambar 4.3: Diagram sekuens dua perjalanan. Pesan yang dikirim logika inti sama persis, dan hanya penerimanya yang berganti

Biaya persiapan basis data karena itu tetap berada di luar, dan logika intinya tidak ikut menanggungnya.

Kasus berikut memperlihatkan betapa mudahnya klaim tersebut gugur. Sebuah tim menyusun domain, port, dan dua adapter dengan rapi. Beberapa waktu kemudian muncul kebutuhan mencatat waktu pembacaan ke berkas log. Seseorang menambahkan satu baris impor pustaka basis data di dalam `domain.py` agar pekerjaan cepat selesai. Susunan berkasnya masih terlihat rapi, dan tidak ada galat yang muncul. Pemeriksaan otomatis kemudian melaporkan satu adapter disebut, dan klaim kemandirian gugur sejak baris itu ditambahkan.

Kejadian tersebut menjelaskan mengapa klaim yang menyebut angka lebih berguna daripada klaim yang hanya menyebut sifat. Kerapian susunan berkas tidak dapat diperiksa mesin, sedangkan jumlah adapter yang disebut dapat.

4.4 Tiga Akibat yang Dapat Diukur

Susunan tersebut membawa tiga akibat, dan ketiganya menghasilkan angka. Tabel 4.3 merangkumnya beserta hasil pengukuran nyata pada contoh bab ini.

Tabel 4.3: Tiga akibat susunan port dan adapter beserta hasil pengukurannya

Akibat	Angka	Cara mengukur
Logika inti tidak mengenal teknologi	0 adapter	Menghitung modul yang diimpor domain.py
Penggantian adapter tidak menyentuh logika inti	1 baris	Mengganti pilihan adapter pada berkas perangkat
Pengujian berjalan tanpa infrastruktur	0 basis data	Memasang adapter tiruan berisi satu metode

Akibat pertama adalah yang paling penting, sebab angkanya dapat membuktikan klaim tersebut salah. Bila domain.py ternyata mengimpor sqlite3, klaim kemandirian gugur pada penerapan tersebut, seberapa pun rapi susunan berkasnya. Listing 4.3 memperlihatkan pemeriksaannya.

```
1 def modul_diimpor(nama_berkas):
2     """Mengumpulkan seluruh nama modul yang diimpor sebuah berkas."""
3     with open(nama_berkas, encoding="utf-8") as berkas:
4         pohon = ast.parse(berkas.read(), filename=nama_berkas)
5         nama = []
6         for simpul in ast.walk(pohon):
7             if isinstance(simpul, ast.Import):
8                 nama.extend(a.name for a in simpul.names)
9             elif isinstance(simpul, ast.ImportFrom) and simpul.module:
10                 nama.append(simpul.module)
11         return nama
```

Listing 4.3: Pemeriksaan kemandirian logika inti, tersedia di sources/chapter04/periksa_port.py

4.5 Kelebihan dan Kekurangan

Tabel 4.4 merangkum untung rugi susunan ini dibandingkan *layered architecture* pada Bab 2.

Kekurangannya terletak pada jumlah berkas. Setiap kebutuhan keluar menambah satu port beserta sekurang-kurangnya satu adapter. Untuk aplikasi kecil dengan satu teknologi penyimpanan yang tidak akan berganti, tambahan tersebut belum tentu sepadan.

Tabel 4.4: Perbandingan ports and adapters dengan layered architecture

Aspek	Layered	Ports and adapters
Arah ketergantungan	Ke bawah, menuju penyimpanan	Ke dalam, menuju logika inti
Pengetahuan logika inti	Mengenal lapisan persistensi	Tidak mengenal teknologi apa pun
Penggantian penyimpanan	Menyentuh lapisan persistensi	Menambah satu adapter baru
Pengujian	Membutuhkan basis data atau tiruannya	Cukup adapter tiruan berisi satu metode
Jumlah berkas	Lebih sedikit	Bertambah oleh port dan adapter

4.6 Ringkasan

Fowler [6] mencatat bahwa pembagian berlapis menempatkan data di lapisan paling bawah, sehingga logika inti tetap berada di atas penyimpanan. Ports and adapters membalik arah ketergantungan yang berlaku pada *layered architecture*. Logika inti tidak lagi memanggil penyimpanan. Logika inti menyatakan kebutuhannya sebagai port, lalu adapter mengisinya. Seluruh panah ketergantungan karena itu menunjuk ke dalam.

Port hanya menyebut kelakuan yang dibutuhkan, bukan teknologi yang menyediakannya. Satu port dapat diisi banyak adapter sekaligus. Pada contoh bab ini, adapter memori dan adapter SQLite mengisi port yang sama, dan keduanya menghasilkan jawaban yang sama tanpa mengubah satu baris pun pada logika inti.

Susunan tersebut hanya bekerja bila logika inti tidak membuat adapternya sendiri. Adapter diserahkan dari luar sebagai argumen, dan pemilihannya dipusatkan pada satu berkas perangkat. Berkas perangkat itulah satu-satunya tempat yang mengetahui seluruh adapter.

Klaim utama pola ini menyebut angka yang pasti, yaitu nol teknologi penyimpanan yang dikenal logika inti. Klaim yang menyebut angka dapat dibuktikan salah oleh pemeriksaan sederhana. Pemeriksaan pada bab ini menunjukkan `domain.py` hanya mengimpor satu modul standar, sehingga klaimnya terpenuhi.

Keuntungan tersebut ada harganya, yaitu jumlah berkas yang bertambah. Setiap kebutuhan keluar menambah satu port beserta adapternya. Keputusan memakai pola ini diambil dengan membandingkan tambahan berkas tersebut terhadap kemungkinan pergantian teknologi di kemudian hari.

4.7 Latihan

Ketiga latihan berikut memakai satu basis kode yang sama di `sources/chapter04/`, tetapi menyoroti tiga masalah yang berbeda. Latihan 1 memeriksa kemandirian logika inti. Latihan 2 mengukur biaya penggantian adapter. Latihan 3 memeriksa pengujian tanpa infrastruktur. Penilaian didasarkan pada kelengkapan tabel hasil dan ketepatan jawaban analisisnya. Hasil setiap mahasiswa dapat berbeda, dan yang dilaporkan adalah pola perbandingannya.

Venv yang sama dipakai kembali, dan pengaktifannya seperti pada Listing 4.4. Latihan bab ini hanya memakai pustaka standar Python beserta SQLite yang sudah menyertainya, sehingga tidak ada pemasangan tambahan. Kedua perintah dijalankan dari akar repositori, lalu perpindahan ke `sources/chapter04/` dilakukan sesudahnya.

Listing 4.4: Mengaktifkan venv sebelum ketiga latihan, skrip tersedia di `sources/siapkan_venv.sh`

```
1 bash sources/siapkan_venv.sh
2 source .venv/bin/activate
```

Latihan 1: Membuktikan Kemandirian Logika Inti

Latihan ini menguji klaim utama pola tersebut, yaitu logika inti tidak mengenal teknologi penyimpanan apa pun. Langkah kerjanya sebagai berikut.

1. Jalankan pemeriksaan seperti pada Listing 4.5, lalu catat baris Adapter disebut.
2. Buka `sources/chapter04/domain.py`, lalu pastikan tidak ada nama teknologi penyimpanan di dalamnya.
3. Tambahkan baris `import sqlite3` pada `domain.py`, lalu jalankan ulang pemeriksaannya dan catat hasilnya.
4. Batalkan penambahan tersebut, lalu pastikan pemeriksaan kembali bersih.

Listing 4.5: Menjalankan pemeriksaan kemandirian

```
1 cd sources/chapter04
2 python periksa_port.py
```

Tabel 4.5 memperlihatkan hasil satu kali pemeriksaan nyata.

Isikan hasil pemeriksaan pada Tabel 4.6.

Jawab dua pertanyaan berikut. Pertama, `domain.py` tetap mengimpor satu modul standar, sehingga jelaskan mengapa impor tersebut tidak melanggar klaim. Kedua, jelaskan mengapa klaim yang menyebut angka lebih berguna daripada klaim yang hanya menyebut sifat.

Tabel 4.5: Contoh hasil Latihan 1 yang sudah terisi

Keadaan berkas	Adapter disebut	Klaim
Apa adanya	0	terpenuhi
Setelah import sqlite3 ditambahkan	1	gagal

Tabel 4.6: Lembar isian Latihan 1

Keadaan berkas	Adapter disebut	Klaim
Apa adanya	...	...
Setelah impor ditambahkan	...	...
Modul yang tetap boleh diimpor	...	

Latihan 2: Mengukur Biaya Penggantian Adapter

Latihan ini membuktikan berapa banyak kode yang berubah ketika teknologi penyimpanan diganti. Langkah kerjanya sebagai berikut.

- 1. Jalankan aplikasi dengan kedua adapter seperti pada Listing 4.6, lalu pastikan hasilnya sama.
- 2. Catat berkas mana saja yang harus dibuka untuk berpindah adapter.
- 3. Tulis satu adapter ketiga yang membaca kurs dari berkas teks biasa.
- 4. Catat jumlah berkas dan baris yang berubah agar adapter ketiga tersebut dapat dipakai.

Listing 4.6: Menjalankan aplikasi dengan dua adapter

```
1 cd sources/chapter04
2 python aplikasi.py memori
3 python aplikasi.py sqlite
```

Tabel 4.7 memperlihatkan hasil satu kali pengukuran nyata.

Tabel 4.7: Contoh hasil Latihan 2 yang sudah terisi

Kegiatan	Berkas berubah	Baris
Berpindah dari memori ke SQLite	0	0
Menambah adapter ketiga	2	22

Isikan hasil pengukuran pada Tabel 4.8.

Tabel 4.8: Lembar isian Latihan 2

Kegiatan	Berkas berubah	Baris
Berpindah antar adapter yang sudah ada	...	...
Menambah adapter ketiga	...	...
Berkas domain.py ikut berubah	...	

Jawab dua pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, apakah `domain.py` ikut berubah ketika adapter ketiga ditambahkan. Kedua, bandingkan angka tersebut dengan biaya penggantian basis data pada Bab 2, lalu jelaskan penyebab perbedaannya.

Latihan 3: Menguji Tanpa Infrastruktur

Latihan ini membuktikan bahwa logika inti dapat diuji tanpa basis data. Langkah kerjanya sebagai berikut.

- 1. Jalankan `python uji_tanpa_basis_data.py`, lalu catat hasil ketiga pengujiannya.
- 2. Buka `sources/chapter04/uji_tanpa_basis_data.py`, lalu hitung jumlah baris kelas `SumberTiruan`.
- 3. Tambahkan satu pengujian baru untuk nominal yang melewati batas wajar.
- 4. Hitung berapa banyak infrastruktur yang disentuh seluruh pengujian tersebut.

Tabel 4.9 memperlihatkan hasil satu kali pengujian nyata.

Tabel 4.9: Contoh hasil Latihan 3 yang sudah terisi

Pengujian	Lulus	Basis data disentuh
Hasil benar	ya	0
Nominal nol ditolak	ya	0
Kurs hilang ditolak	ya	0

Isikan hasil pengujian pada Tabel 4.10.

Tabel 4.10: Lembar isian Latihan 3

Pengujian	Lulus	Basis data disentuh
Hasil benar	...	...
Nominal nol ditolak	...	...
Kurs hilang ditolak	...	...
Nominal melewati batas	...	...
Jumlah baris adapter tiruan	...	

Jawab tiga pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, berapa baris kode yang dibutuhkan untuk menggantikan seluruh basis data pada pengujian. Kedua, jelaskan mengapa jumlah baris tersebut menjadi sedikit. Ketiga, dengan menggabungkan hasil ketiga latihan, tentukan apakah tambahan port dan adapter sepadan untuk aplikasi sekecil ini, lalu sebutkan angka yang menjadi dasar keputusannya.

Bibliografi

- [1] Bank Indonesia. Jisdor, kurs acuan non-usd/idr, dan kurs transaksi. Informasi Kurs, Bank Indonesia, 2026. URL <https://www.bi.go.id/id/fungsi-utama/moneter/informasi-kurs/default.aspx>.
- [2] Alistair Cockburn. Hexagonal architecture (ports and adapters). alistair.cockburn.us, 2005. URL <https://alistair.cockburn.us/hexagonal-architecture/>.
- [3] Django Software Foundation. Faq: General. django appears to be a mvc framework. Django Documentation, 2026. URL <https://docs.djangoproject.com/en/stable/faq/general/>.
- [4] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. URL <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [5] Martin Fowler. Gui architectures. martinowler.com, 2006. URL <https://www.martinfowler.com/eaDev/uiArchs.html>.
- [6] Martin Fowler. Presentation domain data layering. martin-fowler.com, 2015. URL <https://martinfowler.com/bliki/PresentationDomainDataLayering.html>.
- [7] Google. Intents and intent filters. Android Developers, App Architecture Guide, 2026. URL <https://developer.android.com/guide/components/intents-filters>.
- [8] Google. Ui layer. Android Developers, Guide to App Architecture, 2026. URL <https://developer.android.com/topic/architecture/ui-layer>.
- [9] Google. Viewmodel overview. Android Developers, Guide to App Architecture, 2026. URL <https://developer.android.com/topic/libraries/architecture/viewmodel>.
- [10] John Gossman. Introduction to model/view/viewmodel pattern for building wpf apps. Microsoft Developer Blogs, 2005. URL <https://learn.microsoft.com/en-us/archive/blogs/johngossman/>

[introduction-to-modelviewviewmodel-pattern-for-building-wpf-apps](#).

- [11] Glenn E. Krasner and Stephen T. Pope. A cookbook for using the model-view-controller user interface paradigm in smalltalk-80. *Journal of Object-Oriented Programming*, 1(3):26–49, 1988. URL <https://www.semanticscholar.org/paper/028a9f18ec7f66b52ca960f3b899701ddd8637f8>.
- [12] Microsoft. Implementing the microservice application layer using the web api. .NET Microservices Architecture Guide, 2021. URL <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/microservice-application-layer-implementation-web-api>.
- [13] Microsoft. N-tier architecture style. Azure Architecture Center, 2024. URL <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier>.
- [14] Microsoft. Common web application architectures. .NET Application Architecture Guide, 2024. URL <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>.
- [15] Mike Potel. Mvp: Model-view-presenter. the taligent programming model for c++ and java. Technical report, Taligent, Inc., 1996. URL <https://www.wildcrest.com/Potel/Portfolio/mvp.pdf>.
- [16] Redux. Redux prior art. Redux Documentation, 2026. URL <https://redux.js.org/understanding/history-and-design/prior-art>.
- [17] Trygve Reenskaug. Models, views, controllers. Technical report, Xerox Palo Alto Research Center, 1979. URL <https://folk.universitetetioslo.no/trygver/1979/mvc-2/1979-12-MVC.pdf>.
- [18] Mark Richards and Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media, 2020. ISBN 978-1492043454. URL <https://www.oreilly.com/library/view/fundamentals-of-software/9781492043447/>.
- [19] Ruby on Rails. Action controller overview. Ruby on Rails Guides, 2026. URL https://guides.rubyonrails.org/action_controller_overview.html.
- [20] Vue.js. The vue instance. Vue.js Guide versi 1, 2016. URL <https://v1.vuejs.org/guide/instance.html>.